
ML on Big Data

Naive Bayes multinomial
en MapReduce sur Spark

K-NN or Naive Bayes — option Naive Bayes

Abdrahamane MBOUROU CAMARA

Master MIAge SITN
Université Paris-Dauphine

Cours « ML on Big Data » — Prof. D. Colazzo

Juillet 2026

Table des matières

1	Description de la solution adoptée	[Item 1 — 3 pts]	3
1.1	Problème		3
1.2	Données		3
1.3	Architecture logicielle		3
2	Algorithmes conçus et description globale	[Item 2a — 3 pts]	4
2.1	Le modèle Naive Bayes multinomial		4
2.2	Formulation MapReduce (entraînement)		4
2.3	Construction du modèle et broadcast		4
2.4	Garantie de prédictions identiques		5
2.5	Variante catégorielle (jeux tabulaires du papier)		5
3	Commentaires des fragments de code clés	[Item 2b — 3 pts]	5
3.1	Cœur MapReduce, version RDD (<code>src/nb_rdd.py</code>)		5
3.2	Cœur MapReduce, version DataFrames (<code>src/nb_dataframe.py</code>)		5
3.3	Calcul du modèle (<code>src/nb_common.py</code>)		6
3.4	Broadcast du modèle et prédiction		6
3.5	Modèle catégoriel et discrétisation (variante du papier)		6
4	Analyse expérimentale : scalabilité	[Item 3 — 3 pts]	6
4.1	Protocole		7
4.2	Résultats — scalabilité en volume		7
4.3	Résultats — scalabilité en parallélisme		8
4.4	Évaluation détaillée : validation croisée et métriques		8
4.5	Variante catégorielle sur les jeux UCI du papier		9
4.6	Comparaison au papier de référence		9
5	Points forts et points faibles des algorithmes	[Item 4 — 3 pts]	9
6	Conclusion		10
A	Annexe : code source complet	[Item 5 — 4 pts]	10
A.1	<code>src/nb_common.py</code>		10
A.2	<code>src/nb_rdd.py</code>		22
A.3	<code>src/nb_dataframe.py</code>		25
A.4	<code>src/benchmark.py</code>		27
A.5	<code>src/uci_experiment.py</code>		32
A.6	<code>tests/test_smoke.py</code>		35
A.7	<code>data/download_sms_spam.py</code>		37
A.8	<code>data/download_20newsgroups.py</code>		39
A.9	<code>data/download_uci.py</code>		40
B	Annexe : notebook exécuté sur petites données	[Item 5 — 4 pts]	41

Organisation du rapport et correspondance avec le barème

Ce document est structuré pour répondre *explicitement*, section par section, aux cinq items notés de l'énoncé (total 20 points).

Item du barème	Points	Section
1. Description de la solution adoptée	3	§1
2a. Algorithmes conçus + description globale	3	§2
2b. Commentaires des fragments de code clés	3	§3
3. Analyse expérimentale (scalabilité)	3	§4
4. Points forts / points faibles des algorithmes	3	§5
5. Annexe : tout le code + notebook (petites données, E/S commentées)	4	Annexes A–B

1 Description de la solution adoptée

[Item 1 — 3 pts]

1.1 Problème

On implémente le classifieur **Naïve Bayes** selon le paradigme **MapReduce** sur Apache Spark, en s'appuyant sur l'algorithme du papier de référence [1] (Zheng, NDSU, 2014). Le classifieur est codé en **deux versions** (contrainte de l'énoncé), **en Python uniquement** :

- version **RDD** (API bas niveau : `map` / `reduceByKey`) ;
- version **DataFrames** (API relationnelle : `explode` + `groupBy/agg`).

Exigence forte que nous garantissons : les deux versions produisent des **prédictions rigoureusement identiques** (§2.4).

Nous couvrons **deux lois** partageant exactement le même cœur MapReduce (seul le calcul du modèle diffère, §2.5) :

- **multinomial** pour la classification de *texte* (sac de mots) ;
- **catégoriel** pour les jeux *tabulaires* du papier (attribut-valeur), avec discrétisation des attributs continus.

1.2 Données

Jeu	Rôle	Taille	Source
SMS Spam Collection	démo texte / smoke test (petit)	5 574 SMS (ham/spam)	UCI
20 Newsgroups	scalabilité (grand)	18 846 docs, 20 classes	scikit-learn
mushroom, car, nursery, adult	variante catégorielle (papier)	1 7k–32k instances	UCI

Les jeux tabulaires sont exactement ceux du papier de référence, ce qui permet une comparaison directe (§4.6). L'énoncé autorise « datasets of your choice ». Le petit jeu sert au notebook et au smoke test ; le grand jeu sert à l'étude de scalabilité. Pour 20 Newsgroups, en-têtes, pieds de page et citations sont retirés pour ne garder qu'un signal textuel.

1.3 Architecture logicielle

La solution isole dans un module **commun** (`src/nb_common.py`) tout ce qui doit être identique entre les deux versions : tokenisation, vocabulaire, découpage, *construction du modèle* (comptes

→ log-probabilités) et *fonction de prédiction*. Les modules `src/nb_rdd.py` et `src/nb_dataframe.py` ne font que produire, chacun à sa manière (map/reduce), les mêmes *comptes entiers*, puis délèguent au module commun. C’est ce qui rend l’égalité des prédictions vraie *par construction*.

Fichier	Rôle
<code>src/nb_common.py</code>	tokenisation, vocabulaire, split, <code>build_model</code> , <code>predict_indices</code> , <code>get_spark</code>
<code>src/nb_rdd.py</code>	entraînement/prédiction en RDD (<code>flatMap/reduceByKey</code> + broadcast)
<code>src/nb_dataframe.py</code>	idem en DataFrames (<code>explode</code> + <code>groupBy/agg</code> + UDF)
<code>src/benchmark.py</code>	étude de scalabilité → <code>results.csv</code> + graphes PNG
<code>data/download_*.py</code>	récupération SMS Spam / 20 Newsgroups
<code>tests/test_smoke.py</code>	test d’égalité RDD == DataFrame
<code>notebooks/demo.ipynb</code>	démo end-to-end sur petites données + comparaison sklearn

2 Algorithmes conçus et description globale

[Item 2a — 3 pts]

2.1 Le modèle Naive Bayes multinomial

Pour un document d , une classe c , un vocabulaire de taille V et un lissage de Laplace $\alpha = 1$, tous les calculs se font **en logarithme** (pour éviter les sous-débordements numériques) :

$$\log P(c) = \log \left(\frac{N_c}{N} \right), \quad \log P(w \mid c) = \log \left(\frac{\text{count}_{w,c} + \alpha}{\text{total}_c + \alpha V} \right), \quad (1)$$

avec N le nombre de documents d’entraînement, N_c ceux de la classe c , $\text{count}_{w,c}$ les occurrences du mot w en classe c , et $\text{total}_c = \sum_w \text{count}_{w,c}$. La prédiction maximise :

$$\text{score}(d, c) = \log P(c) + \sum_{w \in d} \text{count}_w(d) \log P(w \mid c). \quad (2)$$

Le modèle reproduit exactement `sklearn.naive_bayes.MultinomialNB` ($\alpha = 1$, `fit_prior=True`), ce qui est vérifié empiriquement (§2.4).

2.2 Formulation MapReduce (entraînement)

L’entraînement se ramène à des **comptages** — cas idéal pour MapReduce. Le cœur est $\text{count}_{w,c}$:

- **MAP** : pour chaque occurrence d’un mot w dans un document de classe c , émettre la paire $((c, w), 1)$.
- **REDUCE** : sommer les 1 par clé $(c, w) \rightarrow \text{count}_{w,c}$.

On compte de même N_c (documents par classe) et total_c (tokens par classe). La version RDD exprime le MAP par `flatMap` et le REDUCE par `reduceByKey` ; la version DataFrames exprime le MAP par `explode` (une ligne par mot) et le REDUCE par `groupBy(label, word).count()`.

2.3 Construction du modèle et broadcast

Les deux versions passent les mêmes *comptes entiers* à la fonction commune `build_model`, qui calcule les log-probabilités lissées. Le modèle est stocké de façon **creuse mais équivalente au dense** : $\log P(w \mid c)$ n’est mémorisé que pour les (c, w) de compte non nul, plus une valeur par défaut $\log(\alpha/(\text{total}_c + \alpha V))$ pour les mots de compte nul. Ce modèle est **broadcasté** (sc. `broadcast`) : Spark l’envoie une fois par worker (lecture seule) au lieu de le re-sérialiser à chaque tâche.

2.4 Garantie de prédictions identiques

Les comptes étant entiers (reproductibles au bit près) et la prédiction déterministe (même fonction `predict_indices`), les deux versions donnent des prédictions identiques *par construction*.

Vérifications : le smoke test impose l'égalité RDD/DataFrame ; le notebook obtient **accuracy** = 0.9848 sur SMS Spam avec **RDD** = **DataFrame** = **sklearn** (prédictions identiques) ; le benchmark **assert** l'égalité des accuracy sur chacun des 16 points de mesure.

2.5 Variante catégorielle (jeux tabulaires du papier)

Le papier applique en réalité Naive Bayes à des données *tabulaires catégorielles*. Chaque instance a M attributs ; on estime, pour un attribut A_i de valeur a :

$$\log P(A_i = a \mid c) = \log \left(\frac{\text{count}_{a,i,c} + \alpha}{N_c + \alpha D_i} \right), \quad (3)$$

où D_i est la taille du domaine de l'attribut i . La **seule différence** avec le multinomial est le *dénominateur* (N_c , nombre d'instances de la classe, au lieu du total de tokens). Le **comptage MapReduce est identique** : on encode chaque instance comme un sac de features (attribut, valeur), on émet $((c, \text{feature}), 1)$ et on somme. Seul le calcul du modèle change : dans le code, on branche `build_model_categorical` via le paramètre `model_builder` des fonctions d'entraînement — la prédiction reste `predict_indices`, donc **RDD** = **DataFrame** reste vrai. Ce modèle reproduit `sklearn.naive_bayes.CategoricalNB` (vérifié).

Les attributs **continus** (jeu *adult*) sont d'abord **discrétisés** : un premier job MAP/REDUCE calcule (min, max) par colonne (comme le *PreprocessMapper* du papier), puis on découpe en tranches (binning). C'est la structure à **deux jobs MapReduce** du papier.

3 Commentaires des fragments de code clés

[Item 2b — 3 pts]

3.1 Cœur MapReduce, version RDD (`src/nb_rdd.py`)

Le MAP émet une paire par occurrence de mot ; le REDUCE (`reduceByKey`) somme.

```
# MAP : pour un doc (c, [w1, w2, w1, ...]) on émet ((c, w1), 1), ((c, w2), 1), ...
def emit_word_class(doc):
    c, word_indices = doc
    for w in word_indices:
        yield ((c, w), 1)          # une paire clé/valeur par occurrence

# REDUCE : reduceByKey(add) additionne toutes les occurrences -> count_{w,c}
word_class_counts = dict(
    docs.flatMap(emit_word_class).reduceByKey(add).collect()
)
```

3.2 Cœur MapReduce, version DataFrames (`src/nb_dataframe.py`)

`explode` joue le rôle du MAP (une ligne par mot) ; `groupBy().count()` celui du REDUCE.

```
# MAP : explode(words) -> une ligne (label, word) PAR occurrence de mot
exploded = df.select("label", F.explode("words").alias("word"))
# REDUCE : groupBy(label, word).count() -> count_{w,c}
word_class_counts = {
    (r["label"], r["word"]): r["cnt"]
    for r in exploded.groupBy("label", "word").agg(F.count("*").alias("cnt")).collect()
}
```

3.3 Calcul du modèle (src/nb_common.py)

Passage des comptes entiers aux log-probabilités, avec lissage de Laplace ; factorisé pour être *identique* dans les deux versions.

```
# Dénominateur commun à une classe : total_c + alpha * V
denom = [class_token_totals.get(c, 0) + alpha * vocab_size for c in range(n_classes)]
# Valeur par défaut (mot de compte nul dans la classe) : log(alpha / denom)
log_likelihood_default = [math.log(alpha / denom[c]) for c in range(n_classes)]
# Pour chaque (classe, mot) observé : log((count + alpha) / denom)
for (c, w), count in word_class_counts.items():
    log_likelihood[c][w] = math.log((count + alpha) / denom[c])
# Priors : log P(c) = log(N_c / N)
log_prior[c] = math.log(class_doc_counts[c] / n_docs)
```

3.4 Broadcast du modèle et prédiction

Le modèle est diffusé une fois par worker, puis utilisé en lecture seule (map RDD / UDF DF).

```
# --- RDD ---
bc_model = sc.broadcast(model) # diffusion en lecture seule
preds = docs.map(lambda cw: C.predict_indices(bc_model.value, cw[1])).collect()

# --- DataFrame ---
bc_model = sc.broadcast(model)
@F.udf(returnType=IntegerType())
def predict_udf(words): # closure référençant bc_model.value
    return int(C.predict_indices(bc_model.value, words or []))
```

Remarque technique : `get_spark()` appelle `sc.addPyFile(...)` sur nos modules, sans quoi les workers lèveraient `ModuleNotFoundError` : `nb_common` lors de la désérialisation du modèle et de l'appel à `predict_indices`.

3.5 Modèle catégoriel et discrétisation (variante du papier)

Même comptage, dénominateur différent ($N_c + \alpha D_i$) ; et le job de discrétisation (min/max) est un MAP/REDUCE à part entière.

```
# Modèle CATÉGORIEL : denom dépend de l'attribut de la feature (N_c + alpha * D_i)
for c in range(n_classes):
    n_c = class_doc_counts.get(c, 0)
    for f in range(vocab_size):
        d_i = attr_domain_sizes[feature_attr[f]] # taille du domaine de l'attribut
        cnt = word_class_counts.get((c, f), 0)
        log_likelihood[c][f] = math.log((cnt + alpha) / (n_c + alpha * d_i))

# Discrétisation : 1er job MAP/REDUCE calculant (min, max) par colonne continue
def emit(row):
    for col in continuous_cols:
        yield (col, (float(row[col]), float(row[col]))) # MAP
minmax = sc.parallelize(rows).flatMap(emit) \
    .reduceByKey(lambda a, b: (min(a[0], b[0]), max(a[1], b[1]))) .collect() # REDUCE
```

4 Analyse expérimentale : scalabilité

[Item 3 — 3 pts]

4.1 Protocole

- **Machine** : macOS (Apple Silicon, arm64), 8 cœurs (4 *performance* + 4 *efficiency*). Spark en mode local (`local[k]`).
- **Expérience A** — **volume** : à `local[*]`, réplication de 20 Newsgroups par un facteur $\in \{\times 1, \times 2, \times 4, \times 8\}$.
- **Expérience B** — **parallélisme** : à volume fixe ($\times 8$, ~ 121 k docs), variation de `local[k]`, $k \in \{1, 2, 4, 8\}$.
- **Anti-fuite** : la réplication est faite **après** le découpage train/test, séparément de chaque côté (un document de test ne peut jamais apparaître, même dupliqué, dans l'entraînement).
- **Bruit de mesure** : `--repeat 3` — chaque point mesuré 3 fois, agrégé en moyenne $\pm$ écart-type (barres d'erreur sur les figures).

Commande : `python src/benchmark.py --factors 1 2 4 8 --cores 1 2 4 8 --repeat 3`.

4.2 Résultats — scalabilité en volume

Facteur	n_train	RDD train	DF train	RDD pred	DF pred	Acc.
$\times 1$	15 076	1.81 ± 0.67	3.07 ± 1.25	0.44 ± 0.04	1.39 ± 0.27	0.620
$\times 2$	30 152	1.89 ± 0.15	3.10 ± 0.20	0.68 ± 0.04	2.01 ± 0.19	0.677
$\times 4$	60 304	3.16 ± 0.15	5.11 ± 0.25	1.26 ± 0.03	3.42 ± 0.21	0.704
$\times 8$	120 608	5.27 ± 0.11	8.57 ± 0.09	2.29 ± 0.09	6.91 ± 1.10	0.711

TABLE 1 – Temps (s, moyenne $\pm$ écart-type sur 3 exécutions) selon le volume, à `local[*]`.

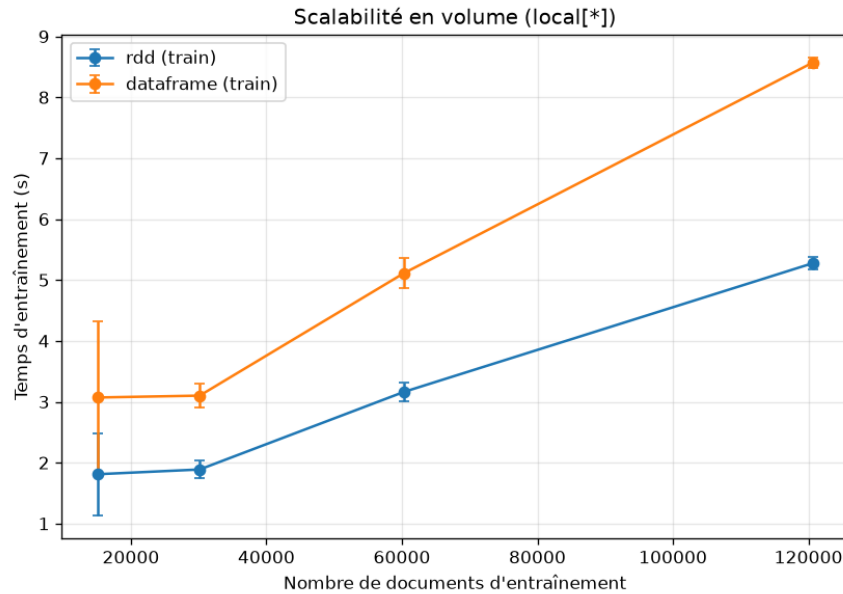


FIGURE 1 – Temps d'entraînement en fonction du volume (`local[*]`).

De $\times 1$ à $\times 8$ ($8\times$ de données), le temps d'entraînement ne croît que d'un facteur ~ 2.9 : **passage à l'échelle sous-linéaire**. RDD est plus rapide que DataFrames à tous les points ; la prédiction DataFrames est $\sim 3\times$ plus lente (UDF Python ligne à ligne).

4.3 Résultats — scalabilité en parallélisme

Cœurs	RDD train	speed-up	RDD pred	DF train	DF pred
1	12.66 ± 0.30	$1.00\times$	5.00 ± 0.04	10.05 ± 0.07	12.07 ± 0.36
2	7.23 ± 0.15	$1.75\times$	3.34 ± 0.08	9.13 ± 0.30	8.63 ± 0.02
4	5.88 ± 0.57	$2.15\times$	2.68 ± 0.28	9.18 ± 0.27	8.09 ± 0.53
8	10.97 ± 1.51	$1.15\times$	3.48 ± 0.51	12.55 ± 1.45	8.30 ± 0.85

TABLE 2 – Temps (s) selon le nombre de cœurs, à volume fixe ($\times 8$). Speed-up = $T(1)/T(k)$.

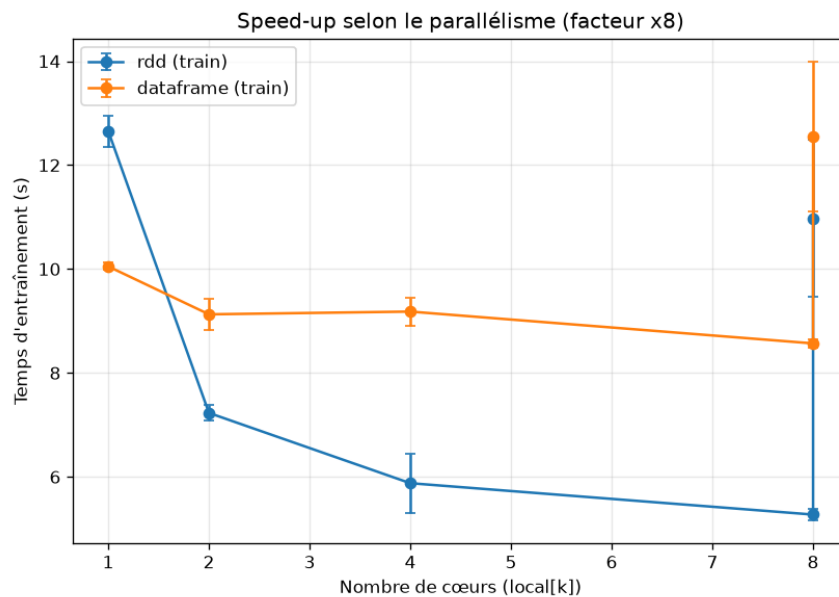


FIGURE 2 – Temps d’entraînement en fonction du nombre de cœurs (facteur $\times 8$).

Le speed-up RDD progresse jusqu’à $\sim 2.15\times$ à 4 cœurs, puis **régresse à 8 cœurs** : sur Apple Silicon, `local[8]` embarque les 4 cœurs *efficiency* (plus lents) et met le driver en contention, d’où un point plus lent et fortement bruité (± 1.5 s). DataFrames ne profite quasiment pas des cœurs supplémentaires.

4.4 Évaluation détaillée : validation croisée et métriques

Au-delà de l’accuracy sur un seul découpage, on suit la méthodologie du papier (§2.3.3–2.3.4) : **validation croisée k-fold** et métriques **précision / rappel / F1** + matrice de confusion. Sur SMS Spam (multinomial), en 5-fold : **accuracy** = 0.9865 ± 0.0029 . La matrice de confusion et les métriques par classe (obtenues dans le notebook) :

Classe	Précision	Rappel	F1
ham	0.986	0.997	0.991
spam	0.978	0.906	0.941
macro	0.982	0.951	0.966

TABLE 3 – Métriques par classe sur SMS Spam (holdout 80/20). Le rappel plus faible du *spam* (0.906) révèle ce que l’accuracy globale (0.985) masque.

4.5 Variante catégorielle sur les jeux UCI du papier

On applique le **Naïve Bayes catégoriel** (avec discrétisation pour *adult*) sur les quatre jeux tabulaires du papier, en **6-fold CV**. À chaque pli, RDD = DataFrames (vérifié par `assert`).

Jeu	Instances	Classes	Accuracy	Macro-F1
mushroom	8 124	2	0.953 ± 0.007	0.953
car	1 728	4	0.858 ± 0.034	0.648
nursery	12 960	5	0.903 ± 0.003	0.568
adult	32 561	2	0.817 ± 0.004	0.769

TABLE 4 – Naïve Bayes catégoriel, 6-fold CV (accuracy moyenne $\pm$ écart-type).

L'écart **accuracy vs macro-F1** (net pour *car* et *nursery*) traduit le **déséquilibre des classes** : la matrice de confusion de *nursery* montre que la classe *recommend* (2 instances sur 12 960) n'est jamais prédite — exactement le scénario où « l'accuracy n'est pas une bonne mesure » décrit au §2.3.3 du papier, d'où l'intérêt d'ajouter précision/rappel/F1.

4.6 Comparaison au papier de référence

- **Algorithme** : notre cœur ($\text{map}((c, \text{feature}), 1) \rightarrow \text{reduce}$ somme, log-vraisemblance, Laplace +1, prior N_c/N) reproduit fidèlement le modèle MapReduce du papier (§3.4). Nous le portons en **Spark RDD et DataFrames / Python**, là où le papier utilise **Hadoop/Java**.
- **Méthodologie** : nous reprenons ses techniques — *augmentation des données* par réplication (§3.3.1), *validation croisée* (§3.3.2), *moyenne sur plusieurs exécutions* avec écart-type (§3.5). Nous ajoutons une correction que le papier ne discute pas : réplication **après** le split (anti-fuite).
- **Scalabilité** : le papier fait varier le nombre de *nœuds* (1 à 14) sur un cluster Hadoop réel ; nous faisons varier les *cœurs local[k]* et le volume sur une seule machine. **Même conclusion qualitative** : gain fort au début puis **rendements décroissants** (lui : 1 nœud 485 s $\rightarrow$ 4 nœuds 180 s $\rightarrow$ 14 nœuds 114 s ; nous : plateau après ~ 4 cœurs).

5 Points forts et points faibles des algorithmes [Item 4 — 3 pts]

Points forts

- **Correction et cohérence** : RDD = DataFrames = sklearn (vérifié) ; calcul en log robuste numériquement.
- **Adéquation MapReduce** : l'entraînement est un pur comptage, parfaitement distribuable ($\text{map} \rightarrow \text{reduceByKey} / \text{groupBy}$).
- **Scalabilité en volume favorable** (sous-linéaire) et **avantage net de la version RDD**, surtout en prédiction.
- **Broadcast** du modèle creux : diffusion unique par worker, économe.
- **Généricité** : les deux lois (multinomial, catégoriel) partagent le même cœur MapReduce ; seul `model_builder` change. Résultats en ligne avec le papier (mushroom 0.95, adult 0.82).

Points faibles

- **Goulot au driver** : les comptes (c, w) sont collectés en mémoire du driver pour construire le modèle broadcasté. Le modèle doit tenir en RAM du driver — c'est la vraie limite «

- big data » (acceptable ici car $V \times C$ reste modéré).
- **Prédiction DataFrames par UDF Python** : sérialisation ligne à ligne, $\sim 3\times$ plus lente que le `map` RDD ; une implémentation par jointures natives serait plus rapide.
 - **Speed-up parallèle limité** en mode local (une seule JVM ; overheads shuffle/GC ; cœurs hétérogènes). Le point `local[8]` est peu fiable.
 - **Effet du lissage sous réplication** : répliquer le train multiplie les comptes par f , si bien que $\log P(w | c) = \log ((f \cdot \text{count} + \alpha) / (f \cdot \text{total} + \alpha V))$; le lissage $\alpha=1$ devient relativement négligeable et le modèle est moins régularisé. Cela explique la légère hausse de l'accuracy ($0.620 \rightarrow 0.711$) : *ce n'est pas une fuite* (le test dupliqué n'apporte aucune information), mais un effet de modélisation. La valeur de référence honnête est celle à $\times 1 = 0.620$.
 - **Hypothèses du NB** : indépendance conditionnelle des attributs (forte), et la **discrétisation** (binning à largeur égale) perd de l'information sur les attributs continus — d'où l'accuracy plus modeste sur *adult* (0.82). Le déséquilibre des classes (*nursery*, *car*) pénalise le macro-F1.

6 Conclusion

Naive Bayes a été implémenté en MapReduce sur Spark en deux versions cohérentes (prédictions identiques, alignées sur scikit-learn) et deux lois (multinomial pour le texte, catégoriel pour les jeux tabulaires du papier), partageant le même cœur MapReduce. L'analyse expérimentale couvre la **scalabilité** (volume et parallélisme), la **validation croisée** et des **métriques** précision/-rappel/F1. Résultats : scaling en volume sous-linéaire, avantage de la version RDD, speed-up parallèle réel mais modéré (limité par le mode local et le matériel), et des accuracies conformes au papier de référence. Pistes : exécution sur cluster, prédiction DataFrames sans UDF Python, métriques par classe, mise à l'échelle du lissage.

Références

- [1] S. Zheng, *Naïve Bayes Classifier : A MapReduce Approach*. Master's paper, North Dakota State University, Fargo, octobre 2014. (Document de référence du cours « ML on Big Data », D. Colazzo, Université Paris-Dauphine.)
- [2] F. Pedregosa *et al.*, *Scikit-learn : Machine Learning in Python*, JMLR 12, 2011. Classe `MultinomialNB`.
- [3] M. Zaharia *et al.*, *Apache Spark : A Unified Engine for Big Data Processing*, CACM 59(11), 2016.
- [4] K. Lang, *NewsWeeder : Learning to Filter Netnews* (jeu 20 Newsgroups), 1995.
- [5] T. Almeida, J. M. Gómez Hidalgo, *SMS Spam Collection*, UCI ML Repository.

A Annexe : code source complet

[Item 5 — 4 pts]

Le code ci-dessous reproduit *tel quel* les fichiers du dépôt (fragment généré depuis les sources par `report/build_notebook_appendix.py`, donc synchronisé avec les fichiers exécutés).

A.1 `src/nb_common.py`

```
"""
nb_common.py
=====
```

Briques **communes** aux deux implémentations de Naive Bayes multinomial (version RDD dans `'nb_rdd.py'` et version DataFrames dans `'nb_dataframe.py'`).

On centralise ici tout ce qui ***doit être strictement identique*** entre les deux versions pour garantir des prédictions rigoureusement égales :

1. la tokenisation (bag-of-words) ;
2. la construction du vocabulaire et de la table des étiquettes ;
3. la vectorisation d'un document en liste d'indices de mots ;
4. la **construction du modèle** à partir de comptes entiers (comptes $\rightarrow$ log-probas) ;
5. la **fonction de prédiction** d'un document à partir du modèle.

Les points 4 et 5 sont volontairement écrits ***une seule fois*** : les versions RDD et DataFrames se contentent de produire, chacune à leur manière (map/reduce), exactement les mêmes comptes entiers, puis appellent ces fonctions communes. Comme les comptes sont des entiers (donc reproductibles au bit près) et que la prédiction est déterministe, les deux versions renvoient des prédictions identiques.

La modélisation reproduit fidèlement `'sklearn.naive_bayes.MultinomialNB'` avec `'alpha=1'` et `'fit_prior=True'` :

$$\begin{aligned}\log P(c) &= \log(N_c / N) \\ \log P(w \mid c) &= \log((\text{count}_{\{w,c\}} + \alpha) / (\text{total}_c + \alpha * V))\end{aligned}$$

où :

N = nombre de documents d'entraînement
 N_c = nombre de documents de la classe c
 $\text{count}_{\{w,c\}}$ = nombre total d'occurrences du mot w dans les docs de classe c
 total_c = $\sum_w \text{count}_{\{w,c\}}$ (nombre total de tokens de la classe c)
 V = taille du vocabulaire
 α = paramètre de lissage de Laplace (1 par défaut)

Score d'un document d pour une classe c (tout en logarithme) :

$$\text{score}(d, c) = \log P(c) + \sum_{w \in d} \text{count}_w(d) * \log P(w \mid c)$$

La classe prédite est celle qui maximise ce score.

"""

```
from __future__ import annotations

import math
import os
import re
import subprocess
from dataclasses import dataclass, field
from typing import Dict, List, Sequence, Tuple

# -----
# 1. Tokenisation (bag-of-words)
# -----
# On reproduit la tokenisation par défaut de sklearn.CountVectorizer :
# - passage en minuscules,
# - motif r"\b\w\w+\b" : suites d'au moins 2 caractères de mot.
# Utiliser exactement la même règle est indispensable pour pouvoir comparer nos
# résultats à ceux de sklearn.MultinomialNB dans le notebook.
_TOKEN_RE = re.compile(r"(?u)\b\w\w+\b")
```

```
def tokenize(text: str) -> List[str]:
    """Découpe un texte en liste de tokens (mots), comme sklearn CountVectorizer.

    >>> tokenize("Free entry, WIN a prize!!!")
    ['free', 'entry', 'win', 'prize']
    """
    if text is None:
        return []
    return _TOKEN_RE.findall(text.lower())

# -----
# 2. Vocabulaire et étiquettes
# -----
def build_vocabulary(
    tokenized_docs: Sequence[Sequence[str]], min_df: int = 1
) -> Dict[str, int]:
    """Construit le vocabulaire {mot -> indice} à partir des documents d'entraînement.

    Le vocabulaire est trié par ordre alphabétique (comme sklearn) afin que les
    indices soient parfaitement déterministes d'une exécution à l'autre.

    Paramètres
    -----
    tokenized_docs : documents déjà tokenisés (liste de listes de mots).
    min_df          : nombre minimal de documents dans lesquels un mot doit
                     apparaître pour être conservé (1 = on garde tout).
    """
    # Compte le nombre de *documents* contenant chaque mot (document frequency).
    doc_freq: Dict[str, int] = {}
    for tokens in tokenized_docs:
        for w in set(tokens): # set() -> un mot compte une seule fois par doc
            doc_freq[w] = doc_freq.get(w, 0) + 1

    # On ne garde que les mots suffisamment fréquents, puis on trie -> indices stables.
    kept = sorted(w for w, df in doc_freq.items() if df >= min_df)
    return {w: i for i, w in enumerate(kept)}

def build_label_index(labels: Sequence[str]) -> Dict[str, int]:
    """Construit la table {étiquette -> entier} (triée pour être déterministe)."""
    return {lab: i for i, lab in enumerate(sorted(set(labels)))}

def doc_to_indices(tokens: Sequence[str], vocab: Dict[str, int]) -> List[int]:
    """Vectorise un document en liste d'indices de mots (avec répétitions).

    Les mots hors-vocabulaire (OOV) sont ignorés : cela correspond exactement au
    comportement de sklearn, où la matrice de features a un nombre fixe de
    colonnes défini au moment du 'fit' sur l'ensemble d'entraînement.

    Exemple : si vocab = {"free": 0, "win": 1} et tokens = ["free", "win", "free", "zzz"],
    on renvoie [0, 1, 0] (le token OOV "zzz" est retiré).
    """
    out: List[int] = []
    for w in tokens:
        idx = vocab.get(w)
        if idx is not None:
            out.append(idx)
```

```

return out

# -----
# 3. Le modèle Naive Bayes (structure partagée)
# -----
@dataclass
class NaiveBayesModel:
    """Modèle Naive Bayes multinomial *dense-équivalent mais stocké creux*.

    Pour respecter le lissage de Laplace, sklearn stocke une log-proba pour
    CHAQUE (classe, mot) du vocabulaire, y compris les mots de compte nul dans
    une classe. Ce serait coûteux (n_classes * V valeurs). On stocke donc :

        - 'log_prior[c]' : log P(c)
        - 'log_likelihood[c][w]' : log P(w|c) uniquement pour les mots dont
                                le compte est NON nul dans la classe c
        - 'log_likelihood_default[c]' : log P(w|c) pour tout mot de compte nul
                                dans la classe c
                                = log( alpha / (total_c + alpha * V) )

    C'est rigoureusement équivalent au calcul dense de sklearn : lors de la
    prédiction, un mot présent dans le document mais de compte nul dans la
    classe utilise simplement la valeur 'default' (voir 'predict_indices').
    Cette représentation creuse est aussi ce que l'on *broadcast* aux workers.
    """

    n_classes: int
    n_docs: int
    vocab_size: int
    alpha: float
    # index entier de classe -> étiquette d'origine (str)
    idx_to_label: List[str]
    log_prior: List[float]
    # log_likelihood[c] : dict {indice_mot -> log P(w|c)} (comptes non nuls seult.)
    log_likelihood: List[Dict[int, float]]
    log_likelihood_default: List[float]

def build_model(
    *,
    n_docs: int,
    vocab_size: int,
    idx_to_label: List[str],
    class_doc_counts: Dict[int, int],
    class_token_totals: Dict[int, int],
    word_class_counts: Dict[Tuple[int, int], int],
    alpha: float = 1.0,
) -> NaiveBayesModel:
    """Construit le modèle (log-probas) à partir de **comptes entiers**.


C'est le cœur du calcul Naive Bayes, factorisé ici pour que les versions RDD
    et DataFrames produisent EXACTEMENT le même modèle : il leur suffit de fournir
    les mêmes agrégats entiers (calculés en map/reduce), le reste est identique.



Paramètres (tous issus d'un comptage MapReduce dans les versions Spark)



-----



|              |   |                                           |                                        |
|--------------|---|-------------------------------------------|----------------------------------------|
| n_docs       | : | N                                         | (nb total de documents d'entraînement) |
| vocab_size   | : | V                                         | (taille du vocabulaire)                |
| idx_to_label | : | correspondance indice_classe -> étiquette |                                        |


```

```

class_doc_counts    : {c -> N_c}          nb de docs par classe
class_token_totals  : {c -> total_c}      nb total de tokens par classe
word_class_counts   : {(c, w) -> count}  nb d'occurrences du mot w en classe c
alpha               : lissage de Laplace (1.0 par défaut)
"""
n_classes = len(idx_to_label)

# --- Priors : log P(c) = log(N_c / N) -----
log_prior = [0.0] * n_classes
for c in range(n_classes):
    n_c = class_doc_counts.get(c, 0)
    # Un prior nul (classe absente) donnerait log(0) = -inf ; en pratique
    # toutes les classes sont présentes en entraînement.
    log_prior[c] = math.log(n_c / n_docs) if n_c > 0 else float("-inf")

# --- Log-vraisemblance par mot : log P(w/c) -----
# Dénominateur commun à une classe : total_c + alpha * V
denom = [class_token_totals.get(c, 0) + alpha * vocab_size for c in range(n_classes)]

# Valeur par défaut (mot de compte nul dans la classe) : log(alpha / denom)
log_likelihood_default = [
    math.log(alpha / denom[c]) if denom[c] > 0 else float("-inf")
    for c in range(n_classes)
]

# Pour chaque (classe, mot) réellement observé : log((count + alpha) / denom)
log_likelihood: List[Dict[int, float]] = [dict() for _ in range(n_classes)]
for (c, w), count in word_class_counts.items():
    log_likelihood[c][w] = math.log((count + alpha) / denom[c])

return NaiveBayesModel(
    n_classes=n_classes,
    n_docs=n_docs,
    vocab_size=vocab_size,
    alpha=alpha,
    idx_to_label=list(idx_to_label),
    log_prior=log_prior,
    log_likelihood=log_likelihood,
    log_likelihood_default=log_likelihood_default,
)

def predict_indices(model: NaiveBayesModel, token_indices: Sequence[int]) -> int:
    """Prédit l'indice de classe d'un document (liste d'indices de mots).

    Fonction PARTAGÉE par les deux versions -> prédictions identiques garanties.

    On calcule, pour chaque classe c :

        score(c) = log P(c) + sum_{w in doc} log P(w|c)

    (chaque occurrence d'un mot ajoute son log P(w|c), ce qui revient bien à
    multiplier par le compte du mot dans le document). On renvoie l'argmax.
    """
    best_c = 0
    best_score = float("-inf")
    for c in range(model.n_classes):
        score = model.log_prior[c]
        ll_c = model.log_likelihood[c]
        default_c = model.log_likelihood_default[c]

```

```

        for w in token_indices:
            # get(w, default) : mot de compte nul dans la classe -> valeur lissée.
            score += ll_c.get(w, default_c)
        if score > best_score:
            best_score = score
            best_c = c
    return best_c

def accuracy(y_true: Sequence[int], y_pred: Sequence[int]) -> float:
    """Exactitude (fraction de prédictions correctes)."""
    if not y_true:
        return 0.0
    correct = sum(1 for a, b in zip(y_true, y_pred) if a == b)
    return correct / len(y_true)

# -----
# 4. Chargement des jeux de données
# -----

def load_sms_spam(csv_path: str) -> Tuple[List[str], List[str]]:
    """Charge le jeu SMS Spam Collection depuis un CSV (colonnes label,text).

    Le fichier est produit par 'data/download_sms_spam.py'.
    Renvoie (textes, étiquettes) avec étiquettes dans {"ham", "spam"}.
    """
    import csv

    texts: List[str] = []
    labels: List[str] = []
    with open(csv_path, "r", encoding="utf-8", newline="") as fh:
        reader = csv.DictReader(fh)
        for row in reader:
            labels.append(row["label"])
            texts.append(row["text"])
    return texts, labels

def load_20newsgroups(
    categories: List[str] | None = None, subset: str = "all"
) -> Tuple[List[str], List[str]]:
    """Charge 20 Newsgroups via scikit-learn (téléchargé/caché automatiquement).

    Renvoie (textes, étiquettes) où l'étiquette est le nom de la catégorie.
    """
    from sklearn.datasets import fetch_20newsgroups

    bunch = fetch_20newsgroups(
        subset=subset,
        categories=categories,
        remove=("headers", "footers", "quotes"), # pour un signal purement textuel
        shuffle=True,
        random_state=42,
    )
    labels = [bunch.target_names[t] for t in bunch.target]
    return list(bunch.data), labels

def replicate_dataset(
    texts: List[str], labels: List[str], factor: int

```

```

) -> Tuple[List[str], List[str]]:
    """Duplique le jeu de données ``factor`` fois (pour l'étude de scalabilité).

    Multiplier la taille des données à distribution constante permet de mesurer
    comment le temps d'exécution évolue avec le *volume*, sans changer l'accuracy.
    """
    if factor <= 1:
        return list(texts), list(labels)
    return texts * factor, labels * factor

# -----
# 5. Découpage entraînement / test
# -----
def train_test_split_texts(
    texts: List[str],
    labels: List[str],
    test_size: float = 0.2,
    seed: int = 42,
) -> Tuple[List[str], List[str], List[str], List[str]]:
    """Découpe (textes, étiquettes) en ensembles d'entraînement et de test.

    On délègue à sklearn pour un découpage stratifié et reproductible.
    Renvoie (X_train, X_test, y_train, y_test).
    """
    from sklearn.model_selection import train_test_split

    # stratify=labels garde les mêmes proportions de classes dans train et test.
    X_train, X_test, y_train, y_test = train_test_split(
        texts, labels, test_size=test_size, random_state=seed, stratify=labels
    )
    return X_train, X_test, y_train, y_test

# -----
# 6. Préparation commune : textes -> (indices de classe, indices de mots)
# -----
@dataclass
class PreparedData:
    """Données prêtes pour l'entraînement Spark, partagées par les deux versions."""

    vocab: Dict[str, int]
    label_to_idx: Dict[str, int]
    idx_to_label: List[str]
    vocab_size: int
    # Chaque exemple : (indice_classe, [indices de mots avec répétitions])
    train: List[Tuple[int, List[int]]]
    test: List[Tuple[int, List[int]]]

def prepare(
    X_train: List[str],
    y_train: List[str],
    X_test: List[str],
    y_test: List[str],
    min_df: int = 1,
) -> PreparedData:
    """Tokenise, construit vocabulaire + étiquettes, et vectorise train/test.

    Cette étape déterministe est faite dans le driver (les données tiennent en

```



```

mémoire ; c'est du prétraitement). Le vocabulaire est construit **uniquement
sur l'entraînement** (comme un 'fit' sklearn), puis appliqué au test.
"""
train_tokens = [tokenize(t) for t in X_train]
test_tokens = [tokenize(t) for t in X_test]

vocab = build_vocabulary(train_tokens, min_df=min_df)
label_to_idx = build_label_index(y_train)
idx_to_label = [lab for lab, _ in sorted(label_to_idx.items(), key=lambda kv: kv[1])]

train = [
    (label_to_idx[lab], doc_to_indices(toks, vocab))
    for lab, toks in zip(y_train, train_tokens)
]
# Les étiquettes de test inconnues à l'entraînement sont improbables ici
# (split stratifié) ; on suppose qu'elles existent dans label_to_idx.
test = [
    (label_to_idx[lab], doc_to_indices(toks, vocab))
    for lab, toks in zip(y_test, test_tokens)
]

return PreparedData(
    vocab=vocab,
    label_to_idx=label_to_idx,
    idx_to_label=idx_to_label,
    vocab_size=len(vocab),
    train=train,
    test=test,
)

# -----
# 7. SparkSession avec le bon JDK
# -----
def _resolve_java_home() -> str | None:
    """Trouve un JAVA_HOME compatible Spark 3.5 (JDK 8/11/17).

    Spark 3.5 ne supporte PAS les JDK plus récents (20, 21, 23...). Sur cette
    machine le 'java' par défaut peut être trop récent : on force donc un JDK 17
    via '/usr/libexec/java_home -v 17' (macOS).
    """
    # Si JAVA_HOME est déjà positionné sur un JDK supporté, on le respecte.
    for version in ("17", "11", "1.8"):
        try:
            out = subprocess.run(
                ["/usr/libexec/java_home", "-v", version],
                capture_output=True,
                text=True,
                check=True,
            )
            path = out.stdout.strip()
            if path and os.path.isdir(path):
                return path
        except Exception:
            continue
    return None

def get_spark(app_name: str = "NaiveBayesMapReduce", master: str = "local[*]"):
    """Crée (ou récupère) une SparkSession configurée pour ce projet.

```

```

- force un JDK compatible (17) si nécessaire ;
- ‘‘master’’ permet de choisir le parallélisme (ex. "local[2]", "local[*]").
"""
java_home = _resolve_java_home()
if java_home:
    os.environ["JAVA_HOME"] = java_home

# Le worker Python doit utiliser le même interpréteur que le driver (venu).
import sys

os.environ.setdefault("PYSPARK_PYTHON", sys.executable)
os.environ.setdefault("PYSPARK_DRIVER_PYTHON", sys.executable)

from pyspark.sql import SparkSession

spark = (
    SparkSession.builder.appName(app_name)
        .master(master)
        # Logs moins verbeux et démarrage plus rapide en local.
        .config("spark.ui.showConsoleProgress", "false")
        .config("spark.sql.shuffle.partitions", "8")
        .getOrCreate()
)

# Les workers exécutent nos fonctions (predict_indices) et désérialisent le
# modèle (dataclass NaiveBayesModel) : ils doivent donc pouvoir importer nos
# modules. On expédie les fichiers source du projet à tous les exécuteurs.
src_dir = os.path.dirname(os.path.abspath(__file__))
for mod in ("nb_common.py", "nb_rdd.py", "nb_dataframe.py"):
    path = os.path.join(src_dir, mod)
    if os.path.isfile(path):
        spark.sparkContext.addPyFile(path)

return spark

# =====
# 8. Métriques d'évaluation (au-delà de l'accuracy)
# =====
# Le papier de référence (Zheng, 2014, §2.3.3) mesure aussi la précision, le
# rappel et la F-mesure, à partir d'une table de contingence / matrice de
# confusion. On les implémente ici (multiclasse, avec moyenne « macro »).
def confusion_matrix(y_true: Sequence[int], y_pred: Sequence[int],
                    n_classes: int) -> List[List[int]]:
    """Matrice de confusion ‘‘M[t][p]’’ = nb d'exemples de vraie classe ‘‘t’’
    prédits en classe ‘‘p’’. La diagonale = prédictions correctes."""
    m = [[0] * n_classes for _ in range(n_classes)]
    for t, p in zip(y_true, y_pred):
        m[t][p] += 1
    return m

def precision_recall_f1(y_true: Sequence[int], y_pred: Sequence[int],
                       n_classes: int) -> dict:
    """Précision, rappel et F1 par classe, plus les moyennes « macro ».


Pour une classe c (vue en « un contre tous ») :


    précision = tp / (tp + fp)    (part des prédicts c qui sont vraiment c)
    rappel    = tp / (tp + fn)    (part des vrais c qu'on retrouve)

```

```

    F1      = 2·P·R / (P + R)  (moyenne harmonique)
    où tp = M[c][c], fp = somme_colonne_c hors diagonale, fn = somme_ligne_c hors diagonale.
    """
    cm = confusion_matrix(y_true, y_pred, n_classes)
    per_class = {}
    precisions, recalls, f1s = [], [], []
    for c in range(n_classes):
        tp = cm[c][c]
        fp = sum(cm[r][c] for r in range(n_classes)) - tp  # prédits c mais pas c
        fn = sum(cm[c][k] for k in range(n_classes)) - tp  # vrais c prédits ailleurs
        prec = tp / (tp + fp) if (tp + fp) > 0 else 0.0
        rec = tp / (tp + fn) if (tp + fn) > 0 else 0.0
        f1 = 2 * prec * rec / (prec + rec) if (prec + rec) > 0 else 0.0
        per_class[c] = {"precision": prec, "recall": rec, "f1": f1, "support": tp + fn}
        precisions.append(prec); recalls.append(rec); f1s.append(f1)
    n = max(n_classes, 1)
    return {
        "per_class": per_class,
        "macro_precision": sum(precisions) / n,
        "macro_recall": sum(recalls) / n,
        "macro_f1": sum(f1s) / n,
        "confusion_matrix": cm,
    }

# =====
# 9. Validation croisée k-fold (§2.3.4 et §3.3.2 du papier)
# =====
def kfold_split(items: Sequence, labels: Sequence, k: int = 5, seed: int = 42):
    """Générateur de découpes k-fold : cède '(X_train, y_train, X_test, y_test)'".

    Le papier fait une validation croisée à 6 plis pour réduire le biais dû à un
    unique découpage. À chaque tour, 1 pli sert de test et les k-1 autres de train.
    On mélange les indices de façon reproductible (seed) avant de les répartir.
    """
    import random

    n = len(items)
    idx = list(range(n))
    random.Random(seed).shuffle(idx)
    # Répartition en k plis de tailles quasi égales.
    folds = [idx[i::k] for i in range(k)]
    for f in range(k):
        test_idx = set(folds[f])
        X_tr = [items[i] for i in idx if i not in test_idx]
        y_tr = [labels[i] for i in idx if i not in test_idx]
        X_te = [items[i] for i in folds[f]]
        y_te = [labels[i] for i in folds[f]]
        yield X_tr, y_tr, X_te, y_te

# =====
# 10. Naive Bayes CATÉGORIEL (attribut-valeur) -- la variante du papier
# =====
# Le papier applique Naive Bayes à des jeux TABULAIRES catégoriels (adult,
# mushroom, nursery...). Chaque instance a M attributs ; on estime
#  $P(A_i = a | c) = (\text{count}_{\{a,i,c\}} + \alpha) / (N_c + \alpha * D_i)$ 
# où  $D_i$  est le nombre de valeurs distinctes de l'attribut  $i$  (domaine), et  $N_c$  le
# nombre d'instances de la classe  $c$ . C'est la différence clé avec le multinomial :
# le dénominateur est  $N_c$  (nb d'instances) et non le total de tokens de la classe.

```

```
# Le comptage map/reduce est IDENTIQUE ; seul le calcul du modèle change.

@dataclass
class PreparedTabular:
    """Données tabulaires prêtes pour l'entraînement (partagé RDD/DataFrame)."""

    vocab: Dict[Tuple[int, str], int] # (indice_attribut, valeur) -> indice de feature
    label_to_idx: Dict[str, int]
    idx_to_label: List[str]
    vocab_size: int
    train: List[Tuple[int, List[int]]] # (classe, [indices de features, 1 par attribut])
    test: List[Tuple[int, List[int]]]
    feature_attr: List[int] # indice de feature -> indice d'attribut
    attr_domain_sizes: List[int] # attribut -> nb de valeurs distinctes (D_i)

def prepare_tabular(X_train: List[List[str]], y_train: List[str],
                    X_test: List[List[str]], y_test: List[str],
                    label_to_idx: Dict[str, int] | None = None) -> PreparedTabular:
    """Vectorise des instances tabulaires catégorielles en indices de features.

    Une « feature » est un couple (attribut, valeur) : on garde les attributs
    distincts même si deux attributs partagent une même valeur. Le vocabulaire et
    les domaines sont construits sur l'ENTRAÎNEMENT seulement (comme un fit).
    Chaque instance devient une liste de M indices (un par attribut).

    'label_to_idx' peut être fourni (mapping GLOBAL des étiquettes) : utile en
    validation croisée pour garder des indices de classe cohérents entre plis,
    même si une classe rare est absente du train d'un pli.
    """
    m = len(X_train[0])
    attr_values = [set() for _ in range(m)]
    for row in X_train:
        for i, v in enumerate(row):
            attr_values[i].add(v)

    # Vocabulaire trié (déterministe) : par attribut puis par valeur.
    vocab: Dict[Tuple[int, str], int] = {}
    feature_attr: List[int] = []
    for i in range(m):
        for v in sorted(attr_values[i]):
            vocab[(i, v)] = len(vocab)
            feature_attr.append(i)
    attr_domain_sizes = [len(attr_values[i]) for i in range(m)]

    if label_to_idx is None:
        label_to_idx = build_label_index(y_train)
    idx_to_label = [lab for lab, _ in sorted(label_to_idx.items(), key=lambda kv: kv[1])]

    def to_indices(row: List[str]) -> List[int]:
        # Une valeur inconnue à l'entraînement (OOV) est ignorée pour cet attribut.
        out = []
        for i, v in enumerate(row):
            idx = vocab.get((i, v))
            if idx is not None:
                out.append(idx)
        return out

    train = [(label_to_idx[l], to_indices(r)) for l, r in zip(y_train, X_train)]
    test = [(label_to_idx[l], to_indices(r)) for l, r in zip(y_test, X_test)]

```

```

    return PreparedTabular(vocab=vocab, label_to_idx=label_to_idx,
                           idx_to_label=idx_to_label, vocab_size=len(vocab),
                           train=train, test=test, feature_attr=feature_attr,
                           attr_domain_sizes=attr_domain_sizes)

def build_model_categorical(
    *,
    n_docs: int,
    vocab_size: int,
    idx_to_label: List[str],
    class_doc_counts: Dict[int, int],
    class_token_totals: Dict[int, int],          # ignoré (garde la même signature)
    word_class_counts: Dict[Tuple[int, int], int],
    alpha: float = 1.0,
    feature_attr: List[int],
    attr_domain_sizes: List[int],
) -> NaiveBayesModel:
    """Construit le modèle Naive Bayes CATÉGORIEL à partir des comptes entiers.

    Même signature que 'build_model' (pour être branché de la même façon dans
    les versions RDD/DataFrame) plus deux métadonnées : 'feature_attr' et
    'attr_domain_sizes'. Reproduit 'sklearn.naive_bayes.CategoricalNB'.

     $\log P(A_i=a \mid c) = \log((\text{count} + \alpha) / (N_c + \alpha * D_i))$ 
    """
    n_classes = len(idx_to_label)

    # Priors :  $\log P(c) = \log(N_c / N)$ .
    log_prior = [
        math.log(class_doc_counts.get(c, 0) / n_docs) if class_doc_counts.get(c, 0) > 0
        else float("-inf")
        for c in range(n_classes)
    ]

    # On stocke DENSE (vocab tabulaire petit) : une log-proba par (classe, feature),
    # y compris les features de compte nul (lissées). Le dénominateur dépend de
    # l'attribut de la feature :  $N_c + \alpha * D_{\text{attr}(feature)}$ .
    log_likelihood: List[Dict[int, float]] = [dict() for _ in range(n_classes)]
    for c in range(n_classes):
        n_c = class_doc_counts.get(c, 0)
        for f in range(vocab_size):
            d_i = attr_domain_sizes[feature_attr[f]]
            cnt = word_class_counts.get((c, f), 0)
            log_likelihood[c][f] = math.log((cnt + alpha) / (n_c + alpha * d_i))

    # Défaut pour une feature hors-vocabulaire au test (valeur jamais vue) : rare ;
    # on suppose un compte nul avec un domaine minimal.
    log_likelihood_default = [
        math.log(alpha / (class_doc_counts.get(c, 0) + alpha))
        if class_doc_counts.get(c, 0) + alpha > 0 else float("-inf")
        for c in range(n_classes)
    ]

    return NaiveBayesModel(
        n_classes=n_classes, n_docs=n_docs, vocab_size=vocab_size, alpha=alpha,
        idx_to_label=list(idx_to_label), log_prior=log_prior,
        log_likelihood=log_likelihood, log_likelihood_default=log_likelihood_default,
    )

```

```

# -----
# 11. Discrétisation des attributs continus (job de prétraitement du papier)
# -----
def spark_minmax(sc, rows: List[List[str]], continuous_cols: List[int]) -> Dict[int, Tuple[float, float]]:
    """Calcule (min, max) de chaque colonne continue via un job MAP/REDUCE.

    Reproduit le PREMIER job MapReduce du papier (PreprocessMapper/Reducer) :
    * MAP : pour chaque colonne continue, émettre (col, (valeur, valeur)) ;
    * REDUCE : reduceByKey en combinant par (min, max).
    """
    def emit(row):
        for c in continuous_cols:
            v = float(row[c])
            yield (c, (v, v))

    def combine(a, b):
        return (min(a[0], b[0]), max(a[1], b[1]))

    res = sc.parallelize(rows).flatMap(emit).reduceByKey(combine).collect()
    return {c: (lo, hi) for c, (lo, hi) in res}

def discretize(rows: List[List[str]], continuous_cols: List[int],
               minmax: Dict[int, Tuple[float, float]], n_bins: int = 5) -> List[List[str]]:
    """Remplace chaque valeur continue par un identifiant de tranche (binning).

    Binning à largeur égale sur [min, max] (bornes issues du train), transformant
    l'attribut continu en attribut catégoriel ('bin0'..'bin{n-1}') exploitable par
    le Naive Bayes catégoriel -- comme le fait le papier (§3.3.3).
    """
    out = []
    for row in rows:
        new = list(row)
        for c in continuous_cols:
            lo, hi = minmax[c]
            v = float(row[c])
            if hi <= lo:
                k = 0
            else:
                k = int((v - lo) / (hi - lo) * n_bins)
                k = min(max(k, 0), n_bins - 1) # borne dans [0, n_bins-1]
            new[c] = f"bin{k}"
        out.append(new)
    return out

```

A.2 src/nb_rdd.py

```

"""
nb_rdd.py
=====

Naive Bayes multinomial **en MapReduce sur des RDD Spark**.

C'est la version « bas niveau » : on manipule directement des RDD et on exprime
l'entraînement comme une suite d'opérations *map* / *reduceByKey*, qui est
exactement l'esprit MapReduce.

```

Idée clé -- comptages par MapReduce :

Un document est (indice_classe, [indices de mots]).

* MAP : pour chaque occurrence d'un mot w dans un doc de classe c , on émet la paire clé/valeur $((c, w), 1)$.

* REDUCE : `reduceByKey(add)` additionne les 1 -> nombre d'occurrences de w dans la classe c , soit `count_{w,c}`.

De même on compte les documents par classe et les tokens par classe.

Ces agrégats entiers sont ensuite passés à `'nb_common.build_model'` (calcul des log-probas), puis le modèle est **broadcasté** aux workers pour la prédiction.

```

from __future__ import annotations

from operator import add
from typing import List, Tuple

import nb_common as C

def train_rdd(sc, train_data: List[Tuple[int, List[int]]], vocab_size: int,
              idx_to_label: List[str], alpha: float = 1.0, num_partitions: int = 8,
              model_builder=C.build_model) -> C.NaiveBayesModel:
    """Entraîne le modèle Naive Bayes en RDD (map/reduceByKey).

    Paramètres
    -----
    sc                : SparkContext.
    train_data        : liste de (indice_classe, [indices de mots]).
    vocab_size        : V, taille du vocabulaire.
    idx_to_label      : correspondance indice_classe -> étiquette d'origine.
    alpha             : lissage de Laplace.
    num_partitions    : nombre de partitions du RDD (parallélisme des données).
    model_builder     : fonction (comptes -> modèle). Par défaut le multinomial
                       'C.build_model' ; passer 'C.build_model_categorical'
                       (via functools.partial) pour la variante catégorielle du
                       papier. Le comptage map/reduce ci-dessous est IDENTIQUE dans
                       les deux cas -- seule la construction du modèle diffère.

    """
    # On distribue les documents sur le cluster (ici local[k]) en 'num_partitions'.
    docs = sc.parallelize(train_data, numSlices=num_partitions)

    # --- (a) Nombre total de documents : N -----
    n_docs = docs.count()

    # --- (b) Nombre de documents par classe : N_c -----
    # MAP : chaque doc -> (classe, 1)
    # REDUCE : reduceByKey(add) -> somme des 1 par classe
    class_doc_counts = dict(
        docs.map(lambda cw: (cw[0], 1)).reduceByKey(add).collect()
    )

    # --- (c) Nombre total de tokens par classe : total_c -----
    # MAP : chaque doc -> (classe, nb de tokens du doc)
    # REDUCE : somme par classe
    class_token_totals = dict(
        docs.map(lambda cw: (cw[0], len(cw[1]))).reduceByKey(add).collect()
    )

    # --- (d) Compte des mots par classe : count_{w,c} (LE cœur MapReduce) ---

```

```

# MAP (flatMap) : pour un doc (c, [w1, w2, w1, ...]), on émet
#                  ((c, w1), 1), ((c, w2), 1), ((c, w1), 1), ...
#                  -> une paire par occurrence de mot.
# REDUCE          : reduceByKey(add) additionne toutes les occurrences,
#                  donnant count_{w,c} pour chaque couple (classe, mot).
def emit_word_class(doc: Tuple[int, List[int]]):
    c, word_indices = doc
    for w in word_indices:
        yield ((c, w), 1)

word_class_counts = dict(
    docs.flatMap(emit_word_class).reduceByKey(add).collect()
)

# --- (e) Construction du modèle (comptes entiers -> log-probas) -----
# Calcul factorisé dans nb_common pour être identique à la version DataFrame.
return model_builder(
    n_docs=n_docs,
    vocab_size=vocab_size,
    idx_to_label=idx_to_label,
    class_doc_counts=class_doc_counts,
    class_token_totals=class_token_totals,
    word_class_counts=word_class_counts,
    alpha=alpha,
)

def predict_rdd(sc, model: C.NaiveBayesModel,
                test_data: List[Tuple[int, List[int]]], num_partitions: int = 8
                ) -> List[int]:
    """Prédit les classes des documents de test, en parallèle sur les RDD.

    Le modèle est **broadcasté** : Spark l'envoie une seule fois à chaque worker
    (au lieu de le sérialiser avec chaque tâche), ce qui est essentiel quand le
    modèle est volumineux (V mots x C classes).
    """
    # BROADCAST : diffusion du modèle en lecture seule à tous les workers.
    bc_model = sc.broadcast(model)

    docs = sc.parallelize(test_data, numSlices=num_partitions)

    # MAP : chaque doc -> classe prédite, via la fonction de prédiction COMMUNE.
    # bc_model.value récupère le modèle local au worker (pas de re-sérialisation).
    preds = docs.map(
        lambda cw: C.predict_indices(bc_model.value, cw[1])
    ).collect()

    bc_model.unpersist()
    return preds

def evaluate_rdd(sc, model: C.NaiveBayesModel,
                 test_data: List[Tuple[int, List[int]]], num_partitions: int = 8
                 ) -> float:
    """Renvoie l'exactitude (accuracy) du modèle sur l'ensemble de test."""
    y_true = [c for c, _ in test_data]
    y_pred = predict_rdd(sc, model, test_data, num_partitions=num_partitions)
    return C.accuracy(y_true, y_pred)

```


A.3 src/nb_dataframe.py

```

"""
nb_dataframe.py
=====

Naive Bayes multinomial **avec l'API DataFrames de Spark**.

Même algorithme que 'nb_rdd.py', mais exprimé avec des opérations
relationnelles de haut niveau ('explode', 'groupBy' / 'agg') au lieu de
map/reduceByKey. Le moteur Catalyst optimise ces opérations.

Correspondance avec la version RDD :
    * 'explode' d'un tableau de mots ≈ la phase MAP (une ligne par mot) ;
    * 'groupBy(...).agg(count/sum)' ≈ la phase REDUCE (reduceByKey).

On produit exactement les mêmes agrégats entiers que la version RDD, puis on
appelle le MÊME 'nb_common.build_model' et la MÊME fonction de prédiction :
les deux versions renvoient donc des prédictions identiques.
Le modèle est **broadcasté** aux workers pour la prédiction via une UDF.
"""

from __future__ import annotations

from typing import List, Tuple

from pyspark.sql import Row
from pyspark.sql import functions as F
from pyspark.sql.types import (
    ArrayType,
    IntegerType,
    StructField,
    StructType,
)

import nb_common as C

def _to_dataframe(spark, data: List[Tuple[int, List[int]]], num_partitions: int):
    """Construit un DataFrame (label:int, words:array<int>) à partir des données.

    Chaque ligne = un document : son indice de classe et son sac de mots (liste
    d'indices de mots, avec répétitions).
    """
    schema = StructType([
        StructField("label", IntegerType(), False),
        StructField("words", ArrayType(IntegerType()), False),
    ])
    rows = [Row(label=c, words=list(w)) for c, w in data]
    return spark.createDataFrame(rows, schema=schema).repartition(num_partitions)

def train_dataframe(spark, train_data: List[Tuple[int, List[int]]], vocab_size: int,
                    idx_to_label: List[str], alpha: float = 1.0, num_partitions: int = 8,
                    model_builder=C.build_model) -> C.NaiveBayesModel:
    """Entraîne le modèle Naive Bayes avec l'API DataFrames (groupBy/agg).

    'model_builder' permet de choisir la loi (multinomial par défaut, ou
    'C.build_model_categorical' pour la variante du papier) : le comptage
    explode/groupBy est identique, seule la construction du modèle change.

```

```

"""
df = _to_dataframe(spark, train_data, num_partitions)
df.cache() # réutilisé plusieurs fois ci-dessous -> on le met en cache

# --- (a) Nombre total de documents : N -----
n_docs = df.count()

# --- (b) Nombre de documents par classe : N_c -----
#   groupBy(label).count() ≈ REDUCE : compte les lignes par classe.
class_doc_counts = {
    r["label"]: r["cnt"]
    for r in df.groupBy("label").agg(F.count("*").alias("cnt")).collect()
}

# --- (c) Nombre total de tokens par classe : total_c -----
#   size(words) = nb de tokens du doc ; on somme par classe.
class_token_totals = {
    r["label"]: r["tot"]
    for r in df.groupBy("label")
        .agg(F.sum(F.size("words")).alias("tot"))
        .collect()
}

# --- (d) Compte des mots par classe : count_{w,c} (cœur MAP/REDUCE) -----
#   MAP : explode(words) -> une ligne (label, word) PAR occurrence de mot
#         (équivalent au flatMap qui émet ((c, w), 1) de la version RDD).
#   REDUCE : groupBy(label, word).count() -> count_{w,c}.
exploded = df.select("label", F.explode("words").alias("word"))
word_class_counts = {
    (r["label"], r["word"]): r["cnt"]
    for r in exploded.groupBy("label", "word")
        .agg(F.count("*").alias("cnt"))
        .collect()
}

df.unpersist()

# --- (e) Construction du modèle (identique à la version RDD) -----
return model_builder(
    n_docs=n_docs,
    vocab_size=vocab_size,
    idx_to_label=idx_to_label,
    class_doc_counts=class_doc_counts,
    class_token_totals=class_token_totals,
    word_class_counts=word_class_counts,
    alpha=alpha,
)

def predict_dataframe(spark, model: C.NaiveBayesModel,
                     test_data: List[Tuple[int, List[int]]], num_partitions: int = 8
                     ) -> List[int]:
    """Prédit les classes de test via une UDF s'appuyant sur le modèle broadcasté."""
    sc = spark.sparkContext

    # BROADCAST : le modèle est diffusé une fois par worker (lecture seule),
    # au lieu d'être capturé/sérialisé pour chaque tâche.
    bc_model = sc.broadcast(model)

    # UDF : applique la fonction de prédiction COMMUNE à chaque sac de mots.

```

```

# La closure référence bc_model.value, résolu localement sur chaque worker.
@F.udf(returnType=IntegerType())
def predict_udf(words):
    return int(C.predict_indices(bc_model.value, words or []))

# On attache un identifiant EXPLICITE (position dans test_data) AVANT tout
# shuffle, afin de pouvoir réordonner les prédictions et les aligner sur
# test_data, quel que soit le repartitionnement effectué par Spark.
schema = StructType([
    StructField("idx", IntegerType(), False),
    StructField("words", ArrayType(IntegerType(), False),
])
rows_in = [Row(idx=i, words=list(w)) for i, (_, w) in enumerate(test_data)]
df = spark.createDataFrame(rows_in, schema=schema).repartition(num_partitions)

df = df.withColumn("pred", predict_udf(F.col("words")))
rows = df.select("idx", "pred").orderBy("idx").collect()

bc_model.unpersist()
return [r["pred"] for r in rows]

def evaluate_dataframe(spark, model: C.NaiveBayesModel,
    test_data: List[Tuple[int, List[int]]], num_partitions: int = 8
) -> float:
    """Renvoie l'exactitude (accuracy) du modèle sur l'ensemble de test."""
    y_true = [c for c, _ in test_data]
    y_pred = predict_dataframe(spark, model, test_data, num_partitions=num_partitions)
    return C.accuracy(y_true, y_pred)

```

A.4 src/benchmark.py

```

"""
benchmark.py
=====

Étude de scalabilité de Naive Bayes multinomial en Spark.

On fait varier deux facteurs et on mesure les temps d'entraînement et de
prédiction pour les deux versions (RDD et DataFrames) :

1. la taille des données : on réplique le jeu 20 Newsgroups plusieurs fois
   (facteurs de réplication), pour observer le passage à l'échelle en volume ;
2. le parallélisme : on fait varier 'local[k]' (nombre de cœurs),
   pour observer l'accélération (speed-up) à volume constant.

Deux précautions méthodologiques importantes :

* Pas de fuite de données : la réplication est faite APRÈS le découpage
  train/test, et séparément sur chaque côté. Ainsi une copie d'un document
  d'entraînement ne peut jamais se retrouver dans le test (et inversement) :
  l'accuracy est donc HONNÊTE (aucune fuite). Elle n'est toutefois pas
  parfaitement constante d'un facteur à l'autre : répliquer l'entraînement
  multiplie les comptes par f, ce qui DILUE le lissage de Laplace (alpha
  devient relativement négligeable devant f*count) et modifie légèrement le
  modèle. C'est un effet de modélisation, pas une fuite ; la valeur de
  référence honnête est celle au facteur x1.

* Bruit de mesure : avec '--repeat N' (N > 1), chaque point est mesuré
  N fois ; on reporte la moyenne et l'écart-type des temps. Cela lisse

```

```

    les exécutions aberrantes (variabilité système, GC de la JVM, etc.).

Sorties :
- 'results/results.csv' : toutes les mesures (moyennes + écarts-types) ;
- 'results/scalability_datasize.png' : temps vs taille des données ;
- 'results/scalability_cores.png' : temps vs nombre de cœurs (local[k]).

Usage :
python src/benchmark.py # configuration par défaut
python src/benchmark.py --quick # version rapide (petits facteurs)
python src/benchmark.py --factors 1 2 4 8 --cores 1 2 4 8 # benchmark complet
python src/benchmark.py --factors 1 2 4 8 --cores 1 2 4 8 --repeat 3 # moyenné
"""

from __future__ import annotations

import argparse
import csv
import os
import statistics
import time
from typing import Callable, List, Tuple

import nb_common as C
import nb_rdd
import nb_dataframe

HERE = os.path.dirname(os.path.abspath(__file__))
ROOT = os.path.dirname(HERE)
RESULTS_DIR = os.path.join(ROOT, "results")

def _timeit(fn: Callable):
    """Exécute 'fn' et renvoie (résultat, temps écoulé en secondes)."""
    t0 = time.perf_counter()
    result = fn()
    return result, time.perf_counter() - t0

def _mean_std(values: List[float]) -> Tuple[float, float]:
    """Renvoie (moyenne, écart-type de population) d'une liste de mesures.

    L'écart-type vaut 0 s'il n'y a qu'une seule mesure ('--repeat 1').
    """
    mean = statistics.mean(values)
    std = statistics.pstdev(values) if len(values) > 1 else 0.0
    return mean, std

def run_once(master: str, factor: int, base_texts: List[str], base_labels: List[str],
             repeat: int = 1, seed: int = 42) -> List[dict]:
    """Mesure un point de benchmark pour un (master, facteur de réplication) donné.

    Méthodologie anti-fuite : on découpe D'ABORD le jeu de base en train/test, PUIS
    on réplique chaque côté séparément. La réplication grossit donc le volume
    (train et test) sans jamais mélanger un document entre les deux ensembles.

    Avec '--repeat > 1', on ré-exécute l'entraînement et la prédiction '--repeat'
    fois sur les MÊMES données (la SparkSession est réutilisée), et on agrège les
    temps en moyenne  $\pm$  écart-type.

```

```

Renvoie une liste de dicts (une ligne par version RDD/DataFrame).
"""
# 1) Découpage AVANT répllication -> pas de fuite de données.
X_tr0, X_te0, y_tr0, y_te0 = C.train_test_split_texts(base_texts, base_labels, seed=seed)
# 2) Répllication INDÉPENDANTE de chaque côté pour faire grossir le volume.
X_tr, y_tr = C.replicate_dataset(X_tr0, y_tr0, factor)
X_te, y_te = C.replicate_dataset(X_te0, y_te0, factor)
data = C.prepare(X_tr, y_tr, X_te, y_te)

# 3) SparkSession avec le parallélisme voulu (local[k]).
spark = C.get_spark(app_name=f"bench-{master}-x{factor}", master=master)
spark.sparkContext.setLogLevel("ERROR")
sc = spark.sparkContext
# Nombre de partitions ~ proportionnel au parallélisme (au moins 4).
parts = max(4, sc.defaultParallelism)

rows: List[dict] = []
try:
    n_train = len(data.train)
    n_test = len(data.test)

    # Accumulateurs de temps sur les "repeat" exécutions.
    t_train_rdd_runs: List[float] = []
    t_pred_rdd_runs: List[float] = []
    t_train_df_runs: List[float] = []
    t_pred_df_runs: List[float] = []
    acc_rdd = acc_df = None

    for _ in range(repeat):
        # --- Version RDD -----
        model_rdd, t_train_rdd = _timeit(
            lambda: nb_rdd.train_rdd(sc, data.train, data.vocab_size,
                                     data.idx_to_label, num_partitions=parts)
        )
        acc_rdd, t_pred_rdd = _timeit(
            lambda: nb_rdd.evaluate_rdd(sc, model_rdd, data.test, num_partitions=parts)
        )

        # --- Version DataFrame -----
        model_df, t_train_df = _timeit(
            lambda: nb_dataframe.train_dataframe(spark, data.train, data.vocab_size,
                                                  data.idx_to_label, num_partitions=parts)
        )
        acc_df, t_pred_df = _timeit(
            lambda: nb_dataframe.evaluate_dataframe(spark, model_df, data.test,
                                                    num_partitions=parts)
        )

        t_train_rdd_runs.append(t_train_rdd)
        t_pred_rdd_runs.append(t_pred_rdd)
        t_train_df_runs.append(t_train_df)
        t_pred_df_runs.append(t_pred_df)

    # Contrôle de cohérence : les deux versions donnent la même accuracy.
    assert abs(acc_rdd - acc_df) < 1e-9, (acc_rdd, acc_df)

    # Agrégation moyenne ± écart-type sur les "repeat" exécutions.
    m_train_rdd, s_train_rdd = _mean_std(t_train_rdd_runs)
    m_pred_rdd, s_pred_rdd = _mean_std(t_pred_rdd_runs)

```

```

m_train_df, s_train_df = _mean_std(t_train_df_runs)
m_pred_df, s_pred_df = _mean_std(t_pred_df_runs)

common = dict(master=master, factor=factor, cores=sc.defaultParallelism,
               repeat=repeat, n_train=n_train, n_test=n_test,
               vocab_size=data.vocab_size)
rows.append(**common, "version": "rdd",
            "train_time_s": round(m_train_rdd, 4), "train_time_std": round(s_train_rdd,
4),
            "predict_time_s": round(m_pred_rdd, 4), "predict_time_std": round(
s_pred_rdd, 4),
            "accuracy": round(acc_rdd, 4)})
rows.append(**common, "version": "dataframe",
            "train_time_s": round(m_train_df, 4), "train_time_std": round(s_train_df,
4),
            "predict_time_s": round(m_pred_df, 4), "predict_time_std": round(s_pred_df,
4),
            "accuracy": round(acc_df, 4)})

print(f" [{master}] x{factor}] train={n_train} (repeat={repeat}) | "
      f"RDD train={m_train_rdd:.2f}±{s_train_rdd:.2f}s pred={m_pred_rdd:.2f}±{
s_pred_rdd:.2f}s | "
      f"DF train={m_train_df:.2f}±{s_train_df:.2f}s pred={m_pred_df:.2f}±{s_pred_df:.2f
}s | "
      f"acc={acc_rdd:.3f}")
finally:
    spark.stop()

return rows

def _plot(results: List[dict]) -> None:
    """Génère les graphiques PNG de scalabilité (avec barres d'erreur = écart-type)."""
    import matplotlib
    matplotlib.use("Agg") # backend non interactif (pas de fenêtre)
    import matplotlib.pyplot as plt

    os.makedirs(RESULTS_DIR, exist_ok=True)

    # --- Graphe 1 : temps d'entraînement vs taille des données (à cores fixés) ---
    # On prend le master ayant le plus grand nombre de mesures par facteur.
    ref_master = max(
        {r["master"] for r in results},
        key=lambda m: len({r["factor"] for r in results if r["master"] == m}),
    )
    subset = [r for r in results if r["master"] == ref_master]
    if len({r["factor"] for r in subset}) >= 2:
        fig, ax = plt.subplots(figsize=(7, 5))
        for version in ("rdd", "dataframe"):
            pts = sorted((r for r in subset if r["version"] == version),
                          key=lambda r: r["n_train"])
            ax.errorbar([r["n_train"] for r in pts], [r["train_time_s"] for r in pts],
                        yerr=[r.get("train_time_std", 0.0) for r in pts],
                        marker="o", capsize=3, label=f"{version} (train)")
        ax.set_xlabel("Nombre de documents d'entraînement")
        ax.set_ylabel("Temps d'entraînement (s)")
        ax.set_title(f"Scalabilité en volume ({ref_master})")
        ax.legend()
        ax.grid(True, alpha=0.3)
        fig.tight_layout()

```

```

fig.savefig(os.path.join(RESULTS_DIR, "scalability_datasize.png"), dpi=120)
plt.close(fig)

# --- Graphe 2 : temps vs nombre de cœurs (à facteur fixé le plus grand) ---
factors_with_multi_cores = [
    f for f in {r["factor"] for r in results}
    if len({r["cores"] for r in results if r["factor"] == f}) >= 2
]
if factors_with_multi_cores:
    ref_factor = max(factors_with_multi_cores)
    subset = [r for r in results if r["factor"] == ref_factor]
    fig, ax = plt.subplots(figsize=(7, 5))
    for version in ("rdd", "dataframe"):
        pts = sorted((r for r in subset if r["version"] == version),
                      key=lambda r: r["cores"])
        ax.errorbar([r["cores"] for r in pts], [r["train_time_s"] for r in pts],
                    yerr=[r.get("train_time_std", 0.0) for r in pts],
                    marker="o", capsize=3, label=f"{version} (train)")
    ax.set_xlabel("Nombre de cœurs (local[k])")
    ax.set_ylabel("Temps d'entraînement (s)")
    ax.set_title(f"Speed-up selon le parallélisme (facteur x{ref_factor})")
    ax.legend()
    ax.grid(True, alpha=0.3)
    fig.tight_layout()
    fig.savefig(os.path.join(RESULTS_DIR, "scalability_cores.png"), dpi=120)
    plt.close(fig)

def main() -> None:
    parser = argparse.ArgumentParser(description="Benchmark de scalabilité Naive Bayes/Spark")
    parser.add_argument("--factors", type=int, nargs="+", default=[1, 2, 4],
                        help="Facteurs de réplication du jeu de données.")
    parser.add_argument("--cores", type=int, nargs="+", default=[1, 2, 4],
                        help="Nombres de cœurs pour local[k].")
    parser.add_argument("--categories", type=str, nargs="+", default=None,
                        help="Sous-ensemble de catégories 20 Newsgroups (défaut: toutes).")
    parser.add_argument("--repeat", type=int, default=1,
                        help="Nombre d'exécutions par point ; les temps sont moyennés "
                             "(moyenne ± écart-type) pour lisser le bruit de mesure.")
    parser.add_argument("--quick", action="store_true",
                        help="Mode rapide : 4 catégories, facteurs [1,2], cores [1,2].")
    args = parser.parse_args()

    if args.repeat < 1:
        parser.error("--repeat doit être >= 1")

    if args.quick:
        args.categories = ["sci.space", "rec.autos", "comp.graphics", "talk.politics.misc"]
        args.factors = [1, 2]
        args.cores = [1, 2]

    print("Chargement de 20 Newsgroups ...")
    base_texts, base_labels = C.load_20newsgroups(categories=args.categories, subset="all")
    print(f" {len(base_texts)} documents, {len(set(base_labels))} classes.")
    print(f" repeat={args.repeat} exécution(s) par point.")

    all_rows: List[dict] = []

    # --- Expérience A : volume croissant à parallélisme maximal (local[*]) ---
    print("\n=== Scalabilité en VOLUME (local[*]) ===")

```

```

for f in args.factors:
    all_rows += run_once("local[*]", f, base_texts, base_labels, repeat=args.repeat)

# --- Expérience B : parallélisme croissant à volume fixe (facteur max) ---
print("\n=== Scalabilité en PARALLÉLISME (facteur fixe) ===")
fixed_factor = max(args.factors)
for k in args.cores:
    all_rows += run_once(f"local[{k}]", fixed_factor, base_texts, base_labels,
                        repeat=args.repeat)

# --- Export CSV -----
os.makedirs(RESULTS_DIR, exist_ok=True)
csv_path = os.path.join(RESULTS_DIR, "results.csv")
fieldnames = ["master", "cores", "factor", "repeat", "version", "n_train", "n_test",
              "vocab_size", "train_time_s", "train_time_std",
              "predict_time_s", "predict_time_std", "accuracy"]
with open(csv_path, "w", newline="", encoding="utf-8") as fh:
    writer = csv.DictWriter(fh, fieldnames=fieldnames)
    writer.writeheader()
    writer.writerows(all_rows)
print(f"\nMesures écrites dans {csv_path}")

# --- Graphiques -----
_plot(all_rows)
print(f"Graphiques écrits dans {RESULTS_DIR}/ (PNG).")

if __name__ == "__main__":
    main()

```

A.5 src/uci_experiment.py

```

"""
uci_experiment.py
=====

Variante **Naive Bayes catégoriel** (celle du papier de référence, Zheng 2014),
appliquée aux jeux tabulaires UCI (mushroom, car, nursery, adult), en Spark, avec :

* **validation croisée k-fold** (comme le papier, §3.3.2) ;
* **discrétisation** des attributs continus via un job MAP/REDUCE (adult, §3.3.3) ;
* **métriques** : accuracy (moyenne  $\pm$  écart-type sur les plis), plus
  précision / rappel / F1 (macro) et matrice de confusion (§2.3.3) ;
* vérification que les versions **RDD et DataFrames** donnent des prédictions
  identiques à chaque pli.

Le comptage MapReduce est le MÊME que pour le texte ('nb_rdd' / 'nb_dataframe')
-- seule la construction du modèle change ('build_model_categorical'), branchée
via le paramètre 'model_builder'.

Usage :
    python src/uci_experiment.py          # tous les jeux, 6 plis
    python src/uci_experiment.py --dataset mushroom --folds 6
"""

from __future__ import annotations

import argparse
import csv

```



```

import functools
import os
import statistics
from typing import List, Tuple

import nb_common as C
import nb_rdd
import nb_dataframe

HERE = os.path.dirname(os.path.abspath(__file__))
ROOT = os.path.dirname(HERE)
UCI_DIR = os.path.join(ROOT, "data", "uci")
RESULTS_DIR = os.path.join(ROOT, "results")

# Configuration de chaque jeu : séparateur, position de l'étiquette, et indices
# (dans la liste d'attributs, étiquette exclue) des attributs CONTINUS à discrétiser.
DATASETS = {
    # mushroom : étiquette en 1re colonne, 22 attributs catégoriels, rien de continu.
    "mushroom": {"label": "first", "continuous": []},
    # car : étiquette en dernière colonne, 6 attributs catégoriels.
    "car": {"label": "last", "continuous": []},
    # nursery : étiquette en dernière colonne, 8 attributs catégoriels.
    "nursery": {"label": "last", "continuous": []},
    # adult : étiquette en dernière colonne, attributs continus à discrétiser :
    #   age(0), fnlwgt(2), education-num(4), capital-gain(10), capital-loss(11),
    #   hours-per-week(12).
    "adult": {"label": "last", "continuous": [0, 2, 4, 10, 11, 12]},
}

def load_uci(name: str) -> Tuple[List[List[str]], List[str], List[int]]:
    """Charge un jeu UCI en (lignes d'attributs, étiquettes, indices continus)."""
    cfg = DATASETS[name]
    path = os.path.join(UCI_DIR, f"{name}.data")
    rows: List[List[str]] = []
    labels: List[str] = []
    with open(path, "r", encoding="utf-8") as fh:
        for line in fh:
            line = line.strip()
            if not line:
                continue
            fields = [f.strip() for f in line.split(",")]
            if cfg["label"] == "first":
                label, attrs = fields[0], fields[1:]
            else:
                label, attrs = fields[-1], fields[:-1]
            rows.append(attrs)
            labels.append(label)
    return rows, labels, cfg["continuous"]

def _run_fold(sc, spark, X_tr, y_tr, X_te, y_te, continuous, n_bins, label_to_idx):
    """Entraîne (RDD + DataFrame) sur un pli et renvoie (y_true, y_pred, acc).

    Les indices de classe sont GLOBAUX (via 'label_to_idx') pour rester cohérents
    entre plis lors de l'agrégation des métriques.
    """
    # Discrétisation des attributs continus via le job MAP/REDUCE (min/max sur le
    # TRAIN uniquement, pour ne pas fuiter d'information du test).
    if continuous:

```

```

minmax = C.spark_minmax(sc, X_tr, continuous)
X_tr = C.discretize(X_tr, continuous, minmax, n_bins=n_bins)
X_te = C.discretize(X_te, continuous, minmax, n_bins=n_bins)

data = C.prepare_tabular(X_tr, y_tr, X_te, y_te, label_to_idx=label_to_idx)
# Le constructeur de modèle catégoriel a besoin des métadonnées d'attributs.
cat_builder = functools.partial(
    C.build_model_categorical,
    feature_attr=data.feature_attr, attr_domain_sizes=data.attr_domain_sizes,
)

m_rdd = nb_rdd.train_rdd(sc, data.train, data.vocab_size, data.idx_to_label,
                        model_builder=cat_builder)
m_df = nb_dataframe.train_dataframe(spark, data.train, data.vocab_size,
                                    data.idx_to_label, model_builder=cat_builder)
p_rdd = nb_rdd.predict_rdd(sc, m_rdd, data.test)
p_df = nb_dataframe.predict_dataframe(spark, m_df, data.test)
assert p_rdd == p_df, "RDD et DataFrame doivent prédire à l'identique"

y_true = [c for c, _ in data.test]
return y_true, p_rdd, C.accuracy(y_true, p_rdd)

def run_dataset(spark, name: str, folds: int, n_bins: int) -> dict:
    """Exécute la validation croisée k-fold sur un jeu et renvoie un dict de mesures."""
    sc = spark.sparkContext
    rows, labels, continuous = load_uci(name)
    label_to_idx = C.build_label_index(labels) # pour agréger les métriques globales
    n_classes = len(label_to_idx)

    fold_accs: List[float] = []
    all_true: List[int] = []
    all_pred: List[int] = []

    for X_tr, y_tr, X_te, y_te in C.kfold_split(rows, labels, k=folds, seed=42):
        y_true, y_pred, acc = _run_fold(sc, spark, X_tr, y_tr, X_te, y_te,
                                       continuous, n_bins, label_to_idx)
        fold_accs.append(acc)
        # Indices de classe GLOBAUX -> agrégation directe des métriques sur tous les plis.
        all_true.extend(y_true)
        all_pred.extend(y_pred)

    mean_acc = statistics.mean(fold_accs)
    std_acc = statistics.pstdev(fold_accs) if len(fold_accs) > 1 else 0.0
    prf = C.precision_recall_f1(all_true, all_pred, n_classes)

    print(f"\n=== {name} ({len(rows)} instances, {n_classes} classes, "
          f"{folds}-fold CV) ===")
    print(f" Accuracy      : {mean_acc:.4f} ± {std_acc:.4f}")
    print(f" Macro P/R/F1   : {prf['macro_precision']:.4f} / "
          f"{prf['macro_recall']:.4f} / {prf['macro_f1']:.4f}")
    print(f" Matrice de confusion (lignes=vrai, colonnes=prédit) :")
    labels_sorted = [lab for lab, _ in sorted(label_to_idx.items(), key=lambda kv: kv[1])]
    for i, rowm in enumerate(prf["confusion_matrix"]):
        print(f"    {labels_sorted[i]:>12s} | {rowm}")

    return {
        "dataset": name, "n_instances": len(rows), "n_classes": n_classes,
        "folds": folds, "acc_mean": round(mean_acc, 4), "acc_std": round(std_acc, 4),
        "macro_precision": round(prf["macro_precision"], 4),
    }

```

```

        "macro_recall": round(prf["macro_recall"], 4),
        "macro_f1": round(prf["macro_f1"], 4),
    }

def main() -> None:
    parser = argparse.ArgumentParser(description="Naive Bayes catégoriel sur jeux UCI")
    parser.add_argument("--dataset", choices=list(DATASETS) + ["all"], default="all")
    parser.add_argument("--folds", type=int, default=6, help="Nombre de plis (CV).")
    parser.add_argument("--bins", type=int, default=5,
                        help="Nombre de tranches pour la discrétisation des continus.")
    args = parser.parse_args()

    names = list(DATASETS) if args.dataset == "all" else [args.dataset]

    spark = C.get_spark(app_name="uci-categorical-nb", master="local[*]")
    spark.sparkContext.setLogLevel("ERROR")
    rows_out = []
    try:
        for name in names:
            rows_out.append(run_dataset(spark, name, args.folds, args.bins))
    finally:
        spark.stop()

    os.makedirs(RESULTS_DIR, exist_ok=True)
    csv_path = os.path.join(RESULTS_DIR, "uci_results.csv")
    fields = ["dataset", "n_instances", "n_classes", "folds", "acc_mean", "acc_std",
             "macro_precision", "macro_recall", "macro_f1"]
    with open(csv_path, "w", newline="", encoding="utf-8") as fh:
        w = csv.DictWriter(fh, fieldnames=fields)
        w.writeheader()
        w.writerows(rows_out)
    print(f"\nRésultats écrits dans {csv_path}")

if __name__ == "__main__":
    main()

```

A.6 tests/test_smoke.py

```

"""
test_smoke.py
=====

Smoke test : vérifie de bout en bout que les DEUX versions (RDD et DataFrames)
- s'entraînent sur un mini-jeu de 20 lignes,
- produisent EXACTEMENT le même modèle et donc la même accuracy.

Exécutable directement ('python tests/test_smoke.py') ou via pytest
('pytest tests/test_smoke.py').
"""

from __future__ import annotations

import os
import sys

# Rendre le dossier src importable (nb_common, nb_rdd, nb_dataframe).
HERE = os.path.dirname(os.path.abspath(__file__))

```

```

SRC = os.path.join(os.path.dirname(HERE), "src")
sys.path.insert(0, SRC)

import nb_common as C          # noqa: E402
import nb_rdd                  # noqa: E402
import nb_dataframe            # noqa: E402

# --- Mini-jeu de données : 20 messages étiquetés spam/ham -----
# Volontairement séparable pour que la prédiction ait du sens sur si peu de data.
MINI_TEXTS = [
    "win a free prize now",
    "free money win cash prize",
    "claim your free lottery prize",
    "congratulations you won free cash",
    "urgent free offer claim now",
    "win big money free entry",
    "free tickets claim your prize",
    "cash prize winner claim now",
    "exclusive free offer win now",
    "you won a free vacation prize",
    "hey are we still meeting today",
    "call me when you get home",
    "lunch tomorrow at noon sounds good",
    "can you send me the report please",
    "happy birthday see you tonight",
    "the meeting is moved to friday",
    "thanks for your help yesterday",
    "i will be late for dinner",
    "did you finish the homework yet",
    "see you at the office monday",
]
MINI_LABELS = (["spam"] * 10) + (["ham"] * 10)

def run() -> None:
    # Démarrage d'une SparkSession locale utilisant tous les cœurs (local[*]).
    spark = C.get_spark(app_name="SmokeTest", master="local[*]")
    spark.sparkContext.setLogLevel("ERROR") # logs Spark discrets
    sc = spark.sparkContext

    try:
        # Découpage train/test reproductible, puis vectorisation commune.
        X_tr, X_te, y_tr, y_te = C.train_test_split_texts(
            MINI_TEXTS, MINI_LABELS, test_size=0.3, seed=0
        )
        data = C.prepare(X_tr, y_tr, X_te, y_te)

        # Entraînement des deux versions sur EXACTEMENT les mêmes données.
        model_rdd = nb_rdd.train_rdd(
            sc, data.train, data.vocab_size, data.idx_to_label
        )
        model_df = nb_dataframe.train_dataframe(
            spark, data.train, data.vocab_size, data.idx_to_label
        )

        # Évaluation.
        acc_rdd = nb_rdd.evaluate_rdd(sc, model_rdd, data.test)
        acc_df = nb_dataframe.evaluate_dataframe(spark, model_df, data.test)

```

```

# Prédiction brut (doivent être identiques ligne à ligne).
pred_rdd = nb_rdd.predict_rdd(sc, model_rdd, data.test)
pred_df = nb_dataframe.predict_dataframe(spark, model_df, data.test)

print(f"Taille vocabulaire : {data.vocab_size}")
print(f"Train / Test      : {len(data.train)} / {len(data.test)}")
print(f"Accuracy RDD      : {acc_rdd:.4f}")
print(f"Accuracy DataFrame : {acc_df:.4f}")
print(f"Prédiction RDD     : {pred_rdd}")
print(f"Prédiction DF      : {pred_df}")

# --- Assertions du smoke test (multinomial) -----
assert acc_rdd == acc_df, (
    f"Accuracies différentes : RDD={acc_rdd} vs DF={acc_df}"
)
assert pred_rdd == pred_df, (
    "Les prédictions ligne à ligne diffèrent entre RDD et DataFrame."
)
print("\nOK (multinomial) : RDD et DataFrame donnent des prédictions identiques.")

# --- Variante CATÉGORIELLE (jeu tabulaire minuscule) -----
import functools

# 6 instances, 2 attributs catégoriels, 2 classes.
rows = [["a", "x"], ["a", "y"], ["b", "x"],
        ["b", "y"], ["a", "x"], ["b", "y"]]
labs = ["p", "p", "n", "n", "p", "n"]
tab = C.prepare_tabular(rows[:4], labs[:4], rows[4:], labs[4:])
cat_builder = functools.partial(
    C.build_model_categorical,
    feature_attr=tab.feature_attr, attr_domain_sizes=tab.attr_domain_sizes,
)
cm_rdd = nb_rdd.train_rdd(sc, tab.train, tab.vocab_size, tab.idx_to_label,
                          model_builder=cat_builder)
cm_df = nb_dataframe.train_dataframe(spark, tab.train, tab.vocab_size,
                                     tab.idx_to_label, model_builder=cat_builder)
cpred_rdd = nb_rdd.predict_rdd(sc, cm_rdd, tab.test)
cpred_df = nb_dataframe.predict_dataframe(spark, cm_df, tab.test)
assert cpred_rdd == cpred_df, (
    "NB catégoriel : prédictions RDD et DataFrame différentes."
)
print("OK (catégoriel) : RDD et DataFrame donnent des prédictions identiques.")
finally:
    spark.stop()

def test_smoke():
    """Point d'entrée pytest."""
    run()

if __name__ == "__main__":
    run()

```

A.7 data/download_sms_spam.py

```

"""
download_sms_spam.py
=====

```

Télécharge le jeu de données ****SMS Spam Collection**** (UCI) et l'exporte en CSV ('data/sms_spam.csv') avec deux colonnes : 'label' (ham/spam) et 'text'.

C'est le PETIT jeu de données, utilisé par le notebook de démonstration et le smoke test. Il ne nécessite pas de cluster.

Usage :

```
python data/download_sms_spam.py
"""

from __future__ import annotations

import csv
import io
import os
import urllib.request
import zipfile

# Miroir officiel UCI de la SMS Spam Collection (fichier zip contenant un TSV).
URL = "https://archive.ics.uci.edu/static/public/228/sms+spam+collection.zip"
# URL de repli (autre miroir historique).
FALLBACK_URL = (
    "https://raw.githubusercontent.com/justmarkham/pycon-2016-tutorial/master/"
    "data/sms.tsv"
)

HERE = os.path.dirname(os.path.abspath(__file__))
OUT_CSV = os.path.join(HERE, "sms_spam.csv")

def _rows_from_zip(raw: bytes):
    """Extrait les lignes (label, text) du zip UCI (fichier 'SMSSpamCollection')."""
    with zipfile.ZipFile(io.BytesIO(raw)) as zf:
        # Le zip contient un fichier tabulé : "<label>\t<message>" par ligne.
        name = next(n for n in zf.namelist() if "SMSSpamCollection" in n)
        with zf.open(name) as fh:
            for line in io.TextIOWrapper(fh, encoding="utf-8"):
                line = line.rstrip("\n")
                if not line:
                    continue
                label, _, text = line.partition("\t")
                yield label.strip(), text

def _rows_from_tsv(raw: bytes):
    """Extrait les lignes (label, text) du miroir TSV de repli."""
    for line in io.StringIO(raw.decode("utf-8")):
        line = line.rstrip("\n")
        if not line:
            continue
        label, _, text = line.partition("\t")
        yield label.strip(), text

def main() -> None:
    print(f"Téléchargement depuis {URL} ...")
    try:
        with urllib.request.urlopen(URL, timeout=60) as resp:
            raw = resp.read()
```

```

        rows = list(_rows_from_zip(raw))
    except Exception as exc: # pragma: no cover - dépend du réseau
        print(f" échec ({exc}); tentative sur le miroir de repli...")
        with urllib.request.urlopen(FALLBACK_URL, timeout=60) as resp:
            raw = resp.read()
        rows = list(_rows_from_tsv(raw))

    # Écriture du CSV normalisé (en-tête label,text).
    with open(OUT_CSV, "w", encoding="utf-8", newline="") as fh:
        writer = csv.writer(fh)
        writer.writerow(["label", "text"])
        writer.writerows(rows)

    n_spam = sum(1 for lab, _ in rows if lab == "spam")
    print(f"OK : {len(rows)} messages écrits dans {OUT_CSV} "
          f"({n_spam} spam / {len(rows) - n_spam} ham).")

if __name__ == "__main__":
    main()

```

A.8 data/download_20newsgroups.py

```

"""
download_20newsgroups.py
=====

Pré-télécharge (met en cache) le jeu de données **20 Newsgroups** via
scikit-learn. C'est le GRAND jeu de données utilisé pour l'étude de scalabilité
('src/benchmark.py').

scikit-learn met les données en cache dans '~/scikit_learn_data' ; ce script
force simplement ce téléchargement une fois pour toutes et affiche un résumé.

Usage :
    python data/download_20newsgroups.py
"""

from __future__ import annotations

from sklearn.datasets import fetch_20newsgroups

def main() -> None:
    print("Téléchargement / mise en cache de 20 Newsgroups via scikit-learn ...")
    # subset="all" = train + test réunis. remove=(...) enlève en-têtes/pieds/citations
    # pour ne garder qu'un signal textuel (évite un sur-apprentissage trivial).
    bunch = fetch_20newsgroups(
        subset="all",
        remove=("headers", "footers", "quotes"),
        shuffle=True,
        random_state=42,
    )
    print(f"OK : {len(bunch.data)} documents, {len(bunch.target_names)} catégories.")
    print("Catégories :")
    for name in bunch.target_names:
        print(f" - {name}")
    print("\nLes données sont en cache (~/.scikit_learn_data) et prêtes pour le benchmark.")

```

```
if __name__ == "__main__":
    main()
```

A.9 data/download_uci.py

```
"""
download_uci.py
=====

Télécharge les jeux de données tabulaires catégoriels de l'UCI utilisés dans le
papier de référence (Zheng, 2014) : **mushroom, car, nursery, adult**.

Ces jeux servent à la variante **Naive Bayes catégoriel** ('src/uci_experiment.py'),
au plus proche du papier. Les fichiers bruts sont mis en cache dans 'data/uci/'.

Usage :
    python data/download_uci.py
"""

from __future__ import annotations

import os
import urllib.request

BASE = "https://archive.ics.uci.edu/ml/machine-learning-databases"
HERE = os.path.dirname(os.path.abspath(__file__))
OUT_DIR = os.path.join(HERE, "uci")

# nom -> URL du fichier brut .data
DATASETS = {
    "mushroom": f"{BASE}/mushroom/agaricus-lepiota.data",
    "car": f"{BASE}/car/car.data",
    "nursery": f"{BASE}/nursery/nursery.data",
    "adult": f"{BASE}/adult/adult.data",
}

def main() -> None:
    os.makedirs(OUT_DIR, exist_ok=True)
    for name, url in DATASETS.items():
        dest = os.path.join(OUT_DIR, f"{name}.data")
        if os.path.exists(dest) and os.path.getsize(dest) > 0:
            print(f" {name:9s} déjà présent -> {dest}")
            continue
        print(f" {name:9s} téléchargement depuis {url} ...")
        with urllib.request.urlopen(url, timeout=60) as resp:
            raw = resp.read()
        with open(dest, "wb") as fh:
            fh.write(raw)
        n = raw.count(b"\n")
        print(f"          OK ({n} lignes) -> {dest}")
    print("Terminé : jeux UCI en cache dans data/uci/.")

if __name__ == "__main__":
    main()
```


B Annexe : notebook exécuté sur petites données [Item 5 — 4 pts]

Reproduction du notebook `notebooks/demo.ipynb` exécuté de bout en bout sur le petit jeu **SMS Spam**, avec ses **entrées** (code) et **sorties** (résultats), et des commentaires sur les entrées/sorties. (Fragment généré par `report/build_notebook_appendix.py`.)

Naïve Bayes multinomial en MapReduce sur Spark — Démonstration

Ce notebook tourne de bout en bout sur les **PETITES** données (SMS Spam), sans cluster

(Spark en mode `local[*]`). Il montre :

1. le chargement et la vectorisation (bag-of-words) ;
2. l'entraînement en **RDD** (`map/reduceByKey`) ;
3. l'entraînement en **DataFrames** (`explode + groupBy/agg + broadcast`) ;
4. la vérification que les **deux versions donnent des prédictions identiques** ;
5. la **comparaison à `sklearn.naive_bayes.MultinomialNB`**.

Prérequis : venv installé (`requirements.txt`) et un JDK 8/11/17. Le petit jeu SMS Spam est téléchargé automatiquement par la première cellule s'il est absent.

0. Configuration : rendre `src/` importable et télécharger les données si besoin

Entrée [1]

```
import os, sys, subprocess

# Racine du projet = dossier parent de notebooks/
ROOT = os.path.abspath(os.path.join(os.getcwd(), ".." if os.path.basename(os.getcwd()) == "
    notebooks" else "."))
SRC = os.path.join(ROOT, "src")
sys.path.insert(0, SRC)

# Télécharger le petit jeu SMS Spam s'il n'existe pas encore.
sms_csv = os.path.join(ROOT, "data", "sms_spam.csv")
if not os.path.exists(sms_csv):
    print("SMS Spam absent -> téléchargement ...")
    subprocess.run([sys.executable, os.path.join(ROOT, "data", "download_sms_spam.py")], check=
        True)
print("Données :", sms_csv, "->", "OK" if os.path.exists(sms_csv) else "MANQUANT")
```

Sortie [1]

```
Données : /Users/abdrahmanecamara/Documents/Dauphine/Projet ML/data/sms_spam.csv -> OK
```

1. Chargement et préparation des données

On charge les SMS (étiquettes `ham/spam`), on découpe en train/test (stratifié, reproductible), puis on **tokenise** et on construit le **vocabulaire** sur l'entraînement.

Chaque document devient une liste d'indices de mots (bag-of-words).

Entrée [2]

```
import nb_common as C

texts, labels = C.load_sms_spam(sms_csv)
print(f"{len(texts)} messages | spam={labels.count('spam')} ham={labels.count('ham')}")

# Découpage train/test reproductible (80/20, stratifié).
X_tr, X_te, y_tr, y_te = C.train_test_split_texts(texts, labels, test_size=0.2, seed=42)

# Tokenisation + vocabulaire + vectorisation en indices de mots (partagé par les 2 versions).
data = C.prepare(X_tr, y_tr, X_te, y_te)
print(f"Vocabulaire : {data.vocab_size} mots")
print(f"Train / Test : {len(data.train)} / {len(data.test)} documents")
print("Classes :", data.idx_to_label)

# Exemple : un document vectorisé (classe, premiers indices de mots)
c0, w0 = data.train[0]
print("Exemple doc -> classe", data.idx_to_label[c0], "| indices mots:", w0[:15], "...")
```

Sortie [2]

```
5574 messages | spam=747 ham=4827
Vocabulaire : 7716 mots
Train / Test : 4459 / 1115 documents
Classes : ['ham', 'spam']
Exemple doc -> classe ham | indices mots: [4917, 947, 4936, 6783, 7376, 6900, 3496, 3446, 3446]
...
```

2. Démarrage de Spark (local, sans cluster)

get_spark sélectionne automatiquement un JDK compatible (17) et expédie nos modules Python aux workers. Le mode local[*] utilise tous les cœurs de la machine.

Entrée [3]

```
spark = C.get_spark(app_name="demo-sms", master="local[*]")
spark.sparkContext.setLogLevel("ERROR")
sc = spark.sparkContext
print("Spark", spark.version, "| parallélisme =", sc.defaultParallelism, "cœurs")
```

Sortie [3]

```
26/07/09 16:03:36 WARN Utils: Your hostname, Macbook-Abdrahamane.local resolves to a loopback
address: 127.0.0.1; using 192.168.0.32 instead (on interface en0)
26/07/09 16:03:36 WARN Utils: Set SPARK_LOCAL_IP if you need to bind to another address
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
26/07/09 16:03:36 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform
... using builtin-java classes where applicable
Spark 3.5.8 | parallélisme = 8 cœurs
```

3. Version RDD — map / reduceByKey

Le cœur MapReduce : pour chaque occurrence d'un mot w dans un document de classe c , on émet $((c, w), 1)$ (**map**), puis **reduceByKey**(add) additionne les occurrences (**reduce**) pour obtenir $\text{count}_{\{w, c\}}$. Le modèle (log-probas) est ensuite construit

puis **broadcasté** pour la prédiction.

Entrée [4]

```
import nb_rdd

model_rdd = nb_rdd.train_rdd(sc, data.train, data.vocab_size, data.idx_to_label)
acc_rdd = nb_rdd.evaluate_rdd(sc, model_rdd, data.test)
print(f"[RDD] accuracy test = {acc_rdd:.4f}")

# Aperçu du modèle : priors log P(c)
for c, lab in enumerate(model_rdd.idx_to_label):
    print(f"  log P({lab}) = {model_rdd.log_prior[c]:.4f}")
```

Sortie [4]

```
[RDD] accuracy test = 0.9848
  log P(ham) = -0.1440
  log P(spam) = -2.0091
```

4. Version DataFrames — explode / groupBy-agg

Même algorithme, exprimé en relationnel : `explode(words)` produit une ligne par mot (`map`), `groupBy(label, word).count()` agrège (`reduce`). La prédiction utilise une UDF s'appuyant sur le modèle **broadcasté**.

Entrée [5]

```
import nb_dataframe

model_df = nb_dataframe.train_dataframe(spark, data.train, data.vocab_size, data.idx_to_label)
acc_df = nb_dataframe.evaluate_dataframe(spark, model_df, data.test)
print(f"[DataFrame] accuracy test = {acc_df:.4f}")
```

Sortie [5]

```
[DataFrame] accuracy test = 0.9848
```

5. Vérification : RDD et DataFrames donnent des prédictions identiques

Entrée [6]

```
pred_rdd = nb_rdd.predict_rdd(sc, model_rdd, data.test)
pred_df = nb_dataframe.predict_dataframe(spark, model_df, data.test)

print("Accuracy identique :", acc_rdd == acc_df, f"({acc_rdd:.4f})")
print("Prédictions identiques (ligne à ligne) :", pred_rdd == pred_df)
assert pred_rdd == pred_df, "Les deux versions doivent prédire exactement pareil !"
print("OK [OK]")
```

Sortie [6]

```
Accuracy identique : True (0.9848)
Prédictions identiques (ligne à ligne) : True
OK [OK]
```

6. Comparaison avec `sklearn.naive_bayes.MultinomialNB`

On entraîne sklearn avec **le même vocabulaire et la même tokenisation**, et le même lissage ($\alpha=1$). Notre implémentation reproduit MultinomialNB : les prédictions doivent coïncider.

Entrée [7]

```
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score

# Même vocabulaire (celui construit sur le train) + même tokenizer que nos versions Spark.
cv = CountVectorizer(tokenizer=C.tokenize, vocabulary=data.vocab,
                    token_pattern=None, lowercase=False)
Xtr = cv.transform(X_tr)
Xte = cv.transform(X_te)

y_tr_idx = [data.label_to_idx[l] for l in y_tr]
y_te_idx = [data.label_to_idx[l] for l in y_te]

clf = MultinomialNB(alpha=1.0)
clf.fit(Xtr, y_tr_idx)
pred_sk = list(clf.predict(Xte))

acc_sk = accuracy_score(y_te_idx, pred_sk)
print(f"[sklearn]    accuracy test = {acc_sk:.4f}")
print(f"[RDD/DF]      accuracy test = {acc_rdd:.4f}")
print(f"Prédictions Spark == sklearn :", [int(p) for p in pred_rdd] == [int(p) for p in pred_sk])
```

Sortie [7]

```
[sklearn]    accuracy test = 0.9848
[RDD/DF]     accuracy test = 0.9848
Prédictions Spark == sklearn : True
```

7. Prédiction sur de nouveaux messages

On applique le modèle RDD (identique au DataFrame) à quelques messages inédits.

Entrée [8]

```
nouveaux = [
    "Congratulations! You have WON a free prize, claim now!!!",
    "Hey, are we still on for lunch tomorrow?",
    "URGENT: your account needs verification, click this link to win cash",
    "Can you send me the slides before the meeting?",
]

# Vectorisation identique au pipeline d'entraînement.
new_indices = [C.doc_to_indices(C.tokenize(t), data.vocab) for t in nouveaux]
for txt, idxs in zip(nouveaux, new_indices):
    c = C.predict_indices(model_rdd, idxs)
    print(f"[{data.idx_to_label[c]:>4}] {txt}")
```

Sortie [8]

```
[spam] Congratulations! You have WON a free prize, claim now!!!
[ ham] Hey, are we still on for lunch tomorrow?
[spam] URGENT: your account needs verification, click this link to win cash
[ ham] Can you send me the slides before the meeting?
```

8. Métriques détaillées : précision, rappel, F1, matrice de confusion

L'accuracy seule masque le comportement par classe (surtout si les classes sont déséquilibrées, comme ham/spam). On calcule donc, comme le papier de référence (Zheng 2014, §2.3.3), la **précision**, le **rappel**, la **F1** et la **matrice de confusion** sur les prédictions de la version RDD.

Entrée [9]

```
y_true = [c for c, _ in data.test]
prf = C.precision_recall_f1(y_true, pred_rdd, len(data.idx_to_label))

print("Matrice de confusion (lignes = vraie classe, colonnes = prédite) :")
for i, row in enumerate(prf["confusion_matrix"]):
    print(f" {data.idx_to_label[i]:>4} | {row}")
print()
for c, lab in enumerate(data.idx_to_label):
    m = prf["per_class"][c]
    print(f" {lab:>4} : précision={m['precision']:.3f}  rappel={m['recall']:.3f}  F1={m['f1']:.3f}")
print(f"\nMacro-F1 : {prf['macro_f1']:.3f}")
```

Sortie [9]

```
Matrice de confusion (lignes = vraie classe, colonnes = prédite) :
  ham | [963, 3]
 spam | [14, 135]

  ham : précision=0.986  rappel=0.997  F1=0.991
 spam : précision=0.978  rappel=0.906  F1=0.941

Macro-F1 : 0.966
```

9. Validation croisée k-fold

Un unique découpage train/test peut être biaisé. Comme le papier (§3.3.2, 6-fold), on évalue en **validation croisée** : on moyenne l'accuracy sur k plis ($\pm$ écart-type). On réutilise la version RDD.

Entrée [10]

```
import statistics

accs = []
for X_tr_f, y_tr_f, X_te_f, y_te_f in C.kfold_split(texts, labels, k=5, seed=42):
    d = C.prepare(X_tr_f, y_tr_f, X_te_f, y_te_f)
    m = nb_rdd.train_rdd(sc, d.train, d.vocab_size, d.idx_to_label)
    accs.append(nb_rdd.evaluate_rdd(sc, m, d.test))

print("Accuracy par pli :", [f"{a:.3f}" for a in accs])
print(f"Accuracy 5-fold : {statistics.mean(accs):.4f} ± {statistics.pstdev(accs):.4f}")
```

Sortie [10]

```
Accuracy par pli : ['0.986', '0.986', '0.989', '0.982', '0.990']
Accuracy 5-fold : 0.9865 ± 0.0029
```

10. Variante du papier : Naive Bayes catégoriel sur un jeu UCI

Le papier applique en réalité Naive Bayes à des jeux **tabulaires catégoriels**

(mushroom, adult...). On en fait une démonstration compacte sur **mushroom** : même

comptage MapReduce, mais modèle `build_model_categorical`

$(P(A_i=a|c) = (\text{count}+\alpha)/(N_c+\alpha \cdot D_i))$, branché via `model_builder`.

On vérifie

là aussi que RDD et DataFrames coïncident.

Entrée [11]

```
import os, subprocess, functools

# Télécharge le petit jeu mushroom si absent (tabulaire, catégoriel).
mush = os.path.join(ROOT, "data", "uci", "mushroom.data")
if not os.path.exists(mush):
    subprocess.run([sys.executable, os.path.join(ROOT, "data", "download_uci.py")], check=True)

from uci_experiment import load_uci
rows, mlabels, continuous = load_uci("mushroom") # 8124 instances, 22 attributs
Xtr, Xte = rows[:6000], rows[6000:]
ytr, yte = mlabels[:6000], mlabels[6000:]

tab = C.prepare_tabular(Xtr, ytr, Xte, yte)
cat_builder = functools.partial(C.build_model_categorical,
                                feature_attr=tab.feature_attr, attr_domain_sizes=tab.attr_domain_sizes)

cm_rdd = nb_rdd.train_rdd(sc, tab.train, tab.vocab_size, tab.idx_to_label, model_builder=
    cat_builder)
cm_df = nb_dataframe.train_dataframe(spark, tab.train, tab.vocab_size, tab.idx_to_label,
    model_builder=cat_builder)
cp_rdd = nb_rdd.predict_rdd(sc, cm_rdd, tab.test)
cp_df = nb_dataframe.predict_dataframe(spark, cm_df, tab.test)

yte_idx = [c for c, _ in tab.test]
print("Classes      :", tab.idx_to_label, "(e=comestible, p=vénéneux)")
print("RDD == DataFrame  :", cp_rdd == cp_df)
print(f"Accuracy (mushroom): {C.accuracy(yte_idx, cp_rdd):.4f}")
prf_m = C.precision_recall_f1(yte_idx, cp_rdd, len(tab.idx_to_label))
print(f"Macro-F1          : {prf_m['macro_f1']:.4f}")
```

Sortie [11]

```
Classes      : ['e', 'p'] (e=comestible, p=vénéneux)
RDD == DataFrame  : True
Accuracy (mushroom): 0.9096
Macro-F1          : 0.8700
```

Entrée [12]

```
spark.stop()
print("SparkSession arrêtée.")
```

Sortie [12]

```
SparkSession arrêtée.
```